

元件建模與驗證

把真實 Metis 晶片的量測，變成可解釋、可外推、誠實標註的延遲方程式——寫給每一個想看懂的人（CS 大學部背景即可）。

2026-06-05 decode-calibrated 由淺入深 · 不漏細節 所有數字皆可從 committed JSON 重產

第 0 章 — 導論：這份報告在做什麼，以及你需要的所有背景

這一章的目的：讓一個有 CS 大學部背景、但沒碰過 CIM / LLM 加速器的人，讀完之後能看懂後面每一章。我們從「我們在解決什麼問題」由上往下，一路講到「你需要哪些基礎概念」。後面每一章都假設你讀過這一章。

0.1 一句話總結這個專案

我們在做一個模擬器，用來預測「一顆還不存在的晶片」上跑大型語言模型（LLM）會有多快、多省電。那顆不存在的晶片是：一個行動 SoC（像手機裡的晶片），裡面除了常見的 CPU、GPU、NPU，還多了一個 CIM 加速器（下面會解釋 CIM 是什麼）。因為這顆晶片還不存在，我們沒辦法直接量測；所以我們用兩塊真實買得到的晶片（Axelera 公司的 Metis 系列）當作「校準的尺」，先量出真實晶片的行為，再用這些量測去校準模擬器。

這份報告是「Phase 1.1」的成果。整個專案分很多階段（見 §0.6）；Phase 1.1 做的事情是：把真實晶片量到的數據，變成一條一條「數學公式」，這樣模擬器就能用公式算延遲，而不是去查一張巨大的表格。

0.2 為什麼需要模擬器？為什麼不直接拿真晶片測？

研究目標是「CIM + 行動 SoC + LLM」這個組合。問題是：

- 這顆「CIM 行動 SoC」還沒有人做出來——它是一個前瞻的架構假設。
- 我們手上的兩塊真 Metis 晶片不能直接拿來研究 LLM：一塊（Metis Alpha）韌體被原廠鎖死、跑不了完整 LLM；另一塊（量產 Metis Card）只能透過封閉的工具鏈跑原廠預編好的 LLM，看不到內部。

所以策略是：把「研究對象」換成一個模擬的晶片，把「真晶片」降級成校準用的 **ground truth**（真值）。真晶片告訴我們「一個 64×64 的矩陣乘法在 CIM 上要花幾微秒」這種基本積木的成本；模擬器再把這些積木組起來，預測整個 LLM 的表現。

類比：你想知道「一棟還沒蓋的大樓」會多重。你沒辦法直接秤它。但你可以去秤「一塊磚」「一根鋼樑」有多重（真晶片量測），再根據設計圖數有幾塊磚幾根樑（模擬器組合），算出整棟大樓的重量。Phase 1.1 就是在「精準量出每種積木的重量，並寫成公式」。

0.3 一分鐘看懂「LLM 推論」

LLM（如 Llama、Qwen）就是一個超大的數學函數，把一段文字（一串 **token**，你可以想成「字」）轉成下一個 **token**。「推論（inference）」就是實際拿模型來生成文字的過程。它分兩個階段：

- 1. **Prefill**（預填）：把你輸入的整段 **prompt**（例如 1000 個 **token**）一次餵進去，讓模型「讀完」。這階段一次處理很多 **token** → 計算量大。
- 2. **Decode**（解碼）：讀完後，模型一次吐一個 **token**，每吐一個就要把目前所有 **token** 再跑一遍模型的一小部分。這階段一次只處理 1 個 **token** → 計算量小，但要反覆把整個模型的權重從記憶體搬出來。

為什麼這個區分超重要？因為兩個階段的瓶頸完全不同：

	PREFILL	DECODE
一次處理幾個 token	很多（整段 prompt）	1 個
瓶頸	算得快不快（compute-bound）	權重搬得快不快（memory-bound）
比喻	一次搬一整車貨（划算）	每次只搬一個包裹，但要跑同樣遠的路

Phase 1.1 把 decode 校得很準（它是行動裝置上最常見、最吃記憶體頻寬的情境）；**prefill** 我們誠實標註為「未驗證」——後面會反覆看到這個誠實邊界。

一個 **token** 內部會跑哪些「運算（op）」？LLM 每一層（**layer**）大致都做這些事，反覆幾十層：

- **matmul**（矩陣乘法）：算 Q/K/V/O 投影、FFN（前饋網路）、最後的 **lm_head**。這是計算量的主體。
- **attention**（注意力）：Q 和 K 做內積、再和 V 相乘，讓模型「看」前面的 **token**。
- **softmax / RMSNorm / RoPE / SwiGLU / residual**：一堆較小的「輔助運算」，做正規化、位置編碼、激活函數、殘差相加等。
- **kv_cache**：把每個 **token** 算出的 K、V 存起來，下個 **token** 才不用重算。

- **embedding**：把 token（一個整數 ID）查表換成一個向量。

Phase 0.1 已經把「每個模型每跑一個 token 會用到哪些 (op, 形狀)」完整列出來了（我們叫它 **op inventory**）；Phase 0.2 算出「每種 op 跑幾次、佔多少計算/記憶體」。資料裡精確分成 9 大類：**matmul**、**attention**、**softmax**、**norm**、**rope**、**ffn**、**residual**、**kv_cache**、**embedding**。

⚠ 一個容易搞混的命名：資料裡的 **ffn** 這一類，指的是 SwiGLU 激活函數（ $\text{silu} + \text{mul}$ 這種逐元素運算），不是 FFN 的矩陣乘法——FFN 的 gate/up/down 三個矩陣乘法是歸在 **matmul** 那一類的（它們都是 weight-stationary，由 CIM 做）。後面看到 **ffn** 別誤會成投影。

這 9 大類 op，後面每一章會對應到某個硬體單元去模型化。

0.4 硬體：CIM 是什麼？為什麼快又有限制？

一般晶片算矩陣乘法，是把資料從記憶體搬到運算單元、算完再搬回去——搬資料很耗時也耗電（這就是有名的「記憶體牆 memory wall」）。

CIM（Compute-In-Memory，記憶體內運算）的點子是：直接在存權重的記憶體裡面做乘加，省掉搬權重的步驟。Metis 用的是數位 SRAM-CIM：把神經網路的權重「釘」在一個 **crossbar**（交叉陣列）上不動，輸入資料流過去就同時完成一整片矩陣乘法。

- CIM 的強項：**weight-stationary**（權重不動）的矩陣乘法，也就是 LLM 的投影/FFN（權重是固定的模型參數）。又快又省電。
- CIM 的弱項：1. 每次呼叫都可能要付一筆固定的「來回成本」（host 和 CIM 之間透過 PCIe 搬資料）。在我們量測的 Alpha 板上這筆固定成本約 911 微秒（ μs ）——但要注意：Alpha 板沒有 on-card DRAM，所以連 decode 都被迫每次走 PCIe、每次付這筆 floor，這是該板的拓樸限制。到了有 on-card DRAM 的量產卡，decode 串流權重時就不是每次都付這筆固定成本（A2 章會詳細說明這個邊界：floor 只對「離散的 host $\leftrightarrow$ device 搬移」收費）。2. **crossbar** 有固定大小（Metis 是 4 個 core、每核 512×512 ，合併有效輸出寬 2048）。比它大的矩陣要切成好幾塊（tile）分次算；比它小的會塞不滿、浪費效率。3. 不擅長 **attention**：attention 的兩個運算元都是「會變動的資料」（不是固定權重），違反 weight-stationary 的前提。所以 attention 要外包（offload）給 GPU/NPU——本期我們用 GPU（Mali）當 offload 對照組實際量測；NPU 是另一個候選，但本期未量測（卡在 issue #13）。

這就是整個專案的核心設計哲學，叫 **CIM-centric**：先繞著 CIM 的限制設計系統，CIM 做它擅長的（投影/FFN 的矩陣乘法），其他單元（GPU/NPU/CPU）當支援層，去做 CIM 不擅長或不該做的事

(attention、softmax、取樣等)。

我們模擬的 SoC 裡有四個計算單元，各自有一章：

單元	角色	在報告裡
CIM (Metis)	主力：weight-stationary 矩陣乘法	A1 (M1)
GPU (Mali)	attention offload、混合精度	A3 (M4-GPU)
NPU (RKNPU2)	另一個 offload 候選	A5 (M4-NPU，本期未量測)
CPU (ARM A76)	小型輔助 op (softmax/norm/取樣...)	A4 (M4-CPU)

加上記憶體系統 (A2 / M2) 和能耗 (A7 / M7)，就是我們要模型化的東西。

0.5 初學者概念工具箱 (後面會一直用到)

這一節把後面會反覆出現的名詞一次講清楚。看不懂某一章時回來查。

矩陣乘法 matmul、GEMV、GEMM。矩陣乘法是 $[M \times K] \times [K \times N] = [M \times N]$ 。

- 當 $M=1$ (一次處理 1 個 token，也就是 decode) 叫 GEMV (矩陣 $\times$ 向量)。
- 當 $M>1$ (一次處理多個 token，也就是 prefill) 叫 GEMM (矩陣 $\times$ 矩陣)。
- 計算量 (FLOPs) $\approx 2 \times M \times K \times N$ 。K、N 是模型的維度 (例如 hidden size 4096)。

INT8 量化。模型權重原本是 16-bit 浮點數 (FP16)。為了更快更省記憶體，Metis 把權重壓成 8-bit 整數 (INT8)，每個權重只佔 1 byte。這很關鍵：權重的「位元組數」 $\approx$ 權重的「參數量」，所以一個 8B (80 億參數) 的模型，每個 token 大約要搬 7.5 GB 的權重。

weight-stationary (權重駐留)。權重固定不動、輸入流過去。CIM 省電的來源。

tile / crossbar / n_cores。Metis 的物理陣列是每核 512 $\times$ 512、共 4 核；模擬器以「n 核」為最小單位 (n 可調，Metis n=4)，合併有效輸出寬 = $n \times 512 = 2048$ 。輸出維度 N 超過有效寬就要分塊 (沿 N 切 $\lfloor N/2048 \rfloor$ 趟)，延遲持續累加。這是 A1 章的核心。

頻寬 (bandwidth) vs 延遲 (latency)。

- 延遲：做一件事花多少時間 (單位： μ s、ms)。
- 頻寬：每秒能搬多少資料 (單位：GB/s)。

- decode 是 memory-bound，所以 decode 速度 $\approx$ 記憶體頻寬 $\div$ 每 token 要搬的權重位元組。我們講的「記憶體牆」約 24 GB/s，這是 LPDDR5 的有效上限（天花板）；真實模型實際達到的串流頻寬會略低於這個天花板（各模型約 16–21 GB/s，後面 Part B 會看到）。所以別直接拿 24 拿來除——要用實際達到的值。

Roofline 與 arithmetic intensity（計算強度）。這是判斷一個運算是「卡在算」還是「卡在搬」的工具。

- 計算強度 = FLOPs $\div$ bytes（每搬 1 byte 能做幾次運算）。
- 強度低 $\rightarrow$ memory-bound（卡在搬資料，例如 decode，強度 ≈ 2 ）。
- 剪度高 $\rightarrow$ compute-bound（卡在算，例如 prefill）。
- 把「達到的吞吐」對「計算強度」畫出來，會看到一條像屋頂的線（roofline），轉折點叫 knee（拐點）。後面 M1 的圖會用到。

throughput（吞吐最）。每秒做幾十億次運算；吞吐 = 運算數 $\div$ 延遲。單位看精度：浮點是 GFLOP/s（如 Mali GPU 的 FP16），整數是 GOP/s（如 Metis CIM 的 INT8）。

什麼叫「把量測擬合成方程式」？我們在最晶片上量了很多點（例如「N=512 時延遲 18.6 μ s、N=1024 時 24.7 μ s...」）。Phase 1.1 的工作是找一條公式 延遲 = f(形狀) 去貼合這些點。好處：模擬器不用存一張巨大的表，還能外推到沒量過的形狀；公式的參數也有物理意義。

什麼叫「measurement vs prediction（量測 vs 預測）」？把「公式算出來的值（prediction）」和「真晶片量到的值（measurement）」畫在一起比較。兩者越接近，代表模型越準。你的要求是每個 component 都要有這個比較——這正是判斷一個 component「校得準不準」的方法。

什麼叫「gate（門檻）」與「held-out（保留驗證）」？

- gate：我們事先訂的「及格線」。Phase 1.1 有兩種門檻（這個區別很重要）：
- 單一元件的公式門檻（ADR-0006）：相對誤差中位數 $\leq 10\%$ 、第 95 百分位 $\leq 20\%$ 。Part A 每個元件用這個。
- 整個 LLM 端到端的門檻（比較寬鬆）：decode tok/s 的相對誤差 $\leq 25\%$ 。Part B 的組合預測用這個（因為端到端疊了很多元件 + 拓模假設，本就該放寬）。
- 相對誤差 = $|預測 - 量測| \div 量測$ 。
- held-out（保留法）：把一部分資料藏起來不拿來擬合，只拿剩下的擬合，再用藏起來的去考公式。如果公式在「沒看過的資料」上也準，代表它是真的學到規律，不是死背。Phase 1.1 兩個層級都用 held-out：元件層（例如 M1 用較小的形狀擬合、再預測較大的形狀）、以及端到端層（只用 1B/3B 模型擬合 decode 速度、再去預測 8B——8B 是「考題」，用上面那條 $\leq 25\%$ 的門檻判定）。

0.6 PHASE 0 / 1 / 2 是什麼？這份報告在哪裡？

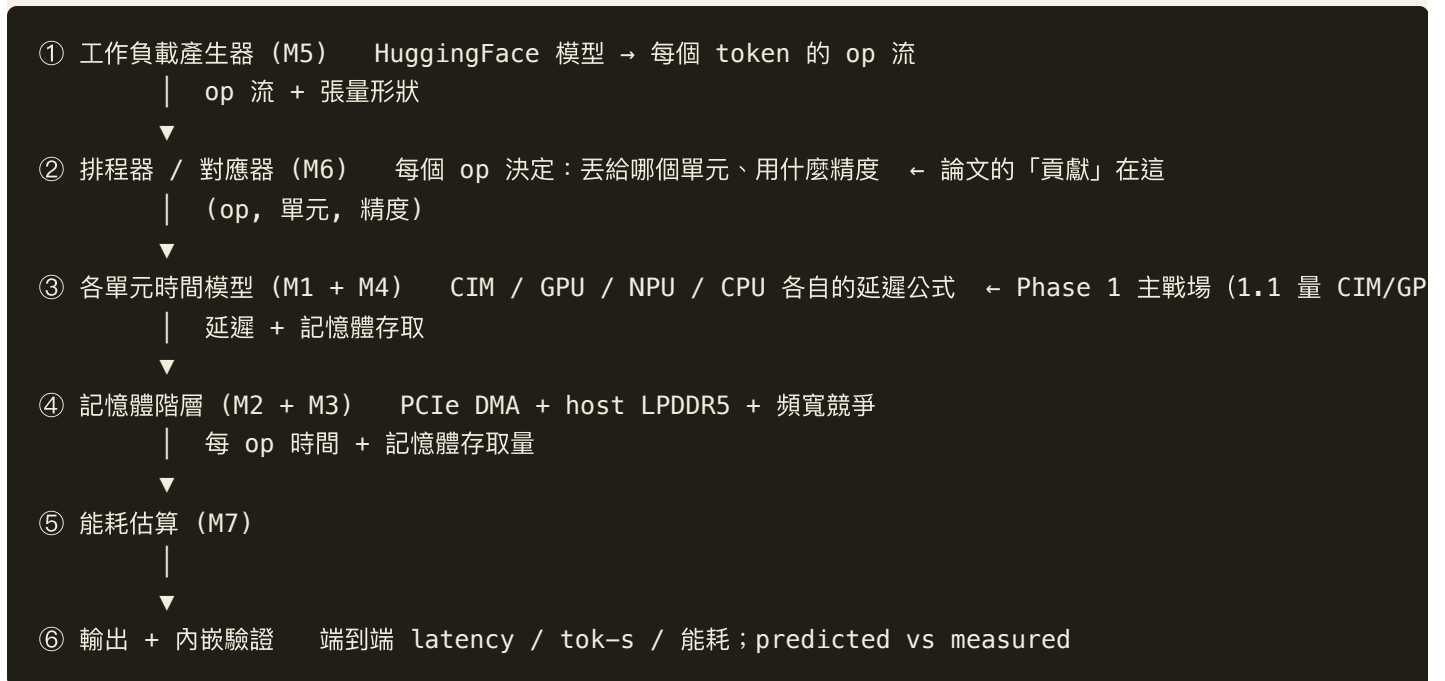
整個專案是一條有先後順序的鏈：



這份報告是 **Phase 1.1**。它的輸入是前面三階段的量測；它的輸出是「一組校好的元件公式 + 參數 + 驗證報告」，給 Phase 2 拿去組裝。

0.7 模擬器的架構：6 個 BOX 與 M1-M7

Phase 2 的模擬器長這樣（資料由上往下流）。Phase 1.1 負責把其中「能直接量測校準」的 box 做成公式：



對應的模組 (module) 編號 M1-M7：

模組	是什麼	PHASE 1.1 做了什麼	報告章節
M1	CIM tile 時間模型	✅ 擬合公式	A1
M2	記憶體階層 (PCIe/LPDDR5/kv_cache)	✅ 擬合 + 解析模型	A2
M3	事件引擎 (把 op 串過各單元)	只定合約 (Phase 2 才實作)	A8
M4	NPU/GPU/CPU 時間模型	✅ GPU/CPU 擬合 ; NPU 待 #13	A3/A4/A5
M5	LLM 工作負載產生器	✅ 驗證 (重用 Phase 0.1)	A6
M6	排程器/對應器 (貢獻層)	只定合約 (Phase 2 才實作)	A8
M7	能耗估算	✅ 規格模型 + 敏感度	A7

為什麼 M3、M6 只定合約不實作？因為它們是「整合層」——要先把各單元的公式 (M1/M4) 和記憶體 (M2) 組起來、真的跑起來，才測得出引擎和排程器對不對。那是 Phase 2 的工作。Phase 1.1 只負責單一元件校準。

0.8 這份報告怎麼讀

全報告分兩大部分：

- **Part A** — 每個元件各一章 (A1–A8)。每一章固定用這個結構，由深到淺：1. 架構考量：這個元件在系統裡扮演什麼角色、為什麼需要它。2. 原理：它背後的硬體/數學原理。3. 參數設計：我們選了什麼形式的公式、有哪些參數、為什麼這樣設計。4. **Measurement vs Prediction**：量測 vs 預測的圖 + 數字（每個元件都有）。5. 圖片解釋：那張圖的每一條軸、每一個點在說什麼。6. 限制與 gap：哪裡沒驗證、為什麼，誠實標註。
- **Part B** — 組合成 LLM。把 Part A 的零件組起來，預測整個 LLM 的 decode 速度 (tok/s)，並比較量測 vs 預測；解釋為什麼 attention 要 offload、能耗為什麼卡在記憶體牆、哪些地方還沒驗證。

讀的時候記住三件事：

1. 「**decode** 校得準、**prefill** 未驗證」是貫穿全報告的誠實邊界。
2. 每個元件都會給你 量測 vs 預測 的對比 + 相對誤差，你可以自己判斷準不準。
3. 我們刻意不藏 gap：哪裡沒量到、哪裡是估的、哪裡是上界/下界，都會明講。一個有 50% 誤差但標清楚的數字，比一個看起來完美但其實循環論證的數字更有價值。

準備好了，就從 **A1：CIM tile**（整個系統的主角）開始。

A1 — M1：CIM tile 時間模型（系統的主角）

這一章你會學到：Metis 的 CIM 在硬體上長什麼樣（四核、每核 512×512 ）、它怎麼算一次矩陣乘法、我們的模擬器為什麼以「 n 個 core」為最小單位、吞吐怎麼隨輸出寬 N 和輸入深 K 變化、以及——哪些是真量測、哪些是我們公式外推出來的（這一版把所有「沒量到卻給數字」的地方都改掉了）。

修訂說明（重要）：這一章對照了 Metis AIPU 的 ISSCC 2024 論文（[筆記](../papers/metis-silicon/metis-aipu-isscc2024.md)）後做了大幅更正。前一版有幾個真錯誤：把「 2048×2048 crossbar」當單一陣列（其實是 $4 \text{ 核} \times 512 \times 512$ ）、用 GFLOP/s（其實是 INT8 的 GOP/s）、把外推點 $N=3072$ 當量測、把 K 效應說成「不可擬合」。以下都修正了。

A1.1 架構考量：CIM 在系統裡是誰？硬體長怎樣？

M1 是 6-box 架構 (§0.7) 裡的 CIM 那一格，是主力計算單元，負責 weight-stationary 矩陣乘法（Q/K/V/O 投影、FFN 的三個 matmul、lm_head）。

硬體（ISSCC 2024 論文）：Metis 是一顆 quad-core（4 個 AI-Core）的晶片，每個 core 內有一塊 512×512 的 INT8 數位 in-memory (D-IMC) crossbar（由 16 個 bank 組成，每 bank = $512 \text{ input} \times 32 \text{ output} \times 4 \text{ weight-set}$ ）。4 核合計峰值 209.6 TOPS。

我們的模擬器以「 n 個 core」為最小單位（ n 是可調參數）：

- 單核原語 = 512×512 。
- n 核合併的有效輸出寬度 = $n \times 512$ 。對 Metis， $n=4 \rightarrow$ 有效寬 2048。
- 我們的量測用的是預設 compile（沒指定 `--aipu-cores`，也沒記下 `cores-spanned`），而觀測到的 tile 邊界剛好 = $2048 = 4 \times 512$ ，這強烈指向預設用了全部 4 個 core（單一 instance 跨 4 核的低延遲映射）。所以我們擬合的公式是「 $n=4$ 的整顆晶片」，不是單核。（嚴格確認 `cores-spanned` 要上板看 `.../<N>/model.json`，但板子目前離線。）

M1 要回答：給形狀 (M, K, N) ，這顆（ $n=4$ ）CIM 算它的 device 計算延遲（ μs ）是多少？（host $\leftrightarrow$ device 的 $911\mu\text{s}$ 來回成本是 M2/A2 的事，不在這裡。）

A1.2 原理：兩個維度怎麼影響速度

decode 時 $M=1$ (GEMV)。一次矩陣乘法 $[1,K] \times [K,N]$ 有兩個維度：

- N = 輸出通道數（要算幾個輸出）。
- K = 輸入深度（每個輸出要累加幾個乘加）。

輸出是「一次 64 個」地產生的（這解釋了「channel-64」這個詞）：論文說整數運算單元每個 cycle 吐 64 個 INT32 輸出，輸出組織成 $8 \times 64 = 512$ 、gating 粒度 32。所以延遲會在 N 為 64 的倍數處有小階梯——這就是「channel-64 staircase」。我們特意測了 3 個非-64 的 N (480/544/1000) 去驗證對齊罰則：結果它們落在曲線上、沒有明顯懲罰，代表 64-量化效應很輕微。

有效吞吐 G_{eff} 同時隨 N 和 K 上升（不是只看 N ）。直覺： N 大、 K 大都讓引擎「填得更滿」、固定 overhead 被攤平，吞吐就高。實測 (INT8、GOP/s)：

固定 N	$K=2048$	$K=3072$	$K=3584$	$K=4096$
$N=512$	112.6	—	147.4	—
$N=1024$	169.8	182.3	—	227.2

→ 同一個 N 、 K 越寬、吞吐越高（這就是後面要 fit 的「 K 效應」）。

device envelope（能配置多大）——為什麼是 6M，不是 32 MB L2？實測探出 $K \cdot N \leq 6M$ 參數可配置（6.3M OK、8.4M 失敗 `zeMemAllocDevice`）。這個 6M 既不是 32 MB L2、也不是 52 MiB 片上 SRAM——關鍵在於 Alpha 板沒有真正的 on-card DRAM：

- `zeMemAllocDevice` 配置的「device 記憶體」，在 Alpha 上其實是一塊把 host LPDDR 映射進裝置位址空間的 PCIe-IOMMU window（不是真 DRAM；`voyager-sdk.md` [MEASURED]）。
- 而這個 window 預設只有 ~14 MB（Axelera 論壇 SMMU-v3 thread #1330，可經 device-tree 擴到 128 MB / 1 GB；`voyager-sdk.md:248`）。
- 6.3M 參數（ ≈ 6 MB 權重）+ 活化 / workspace 剛好逼近這 ~14 MB → 8.4M 就配不下。

所以 $6M \approx$ 「預設 IOMMU window 的權重份額」，跟 L2 (32 MB，weights home 預設、 >32 MB 才 spill) 或晶片 SRAM 容量無關。（嚴格確認那次 run 的 window 大小，要看板上的 `compile_config.json` / device-tree——板子目前離線，這條列為證據鏈到此、待上板補確認。）

A1.3 參數設計：公式與 2D 擬合

延遲公式（ simulator/models/m1_cim_tile.py ）：

```
W = n_cores × 512                                # 有效輸出寬度 (n=4 → 2048)
dev_lat(M,K,N) = Σ over 輸出 tiles (每塊寬 ≤ W)  2·M·K·n_tile / G_eff(n_tile, K)
G_eff(N,K) = Gmax · N/(N+Na) · K/(K+Kb)          # 2D 有效吞吐 (GOP/s)
```

- 2D **G_eff(N,K)**：在 13 個原生單-tile 點上擬合，得 Gmax=333.7 GOP/s、Na=577、Kb=574。這條公式同時吃進 N 和 K——所以前一版那個「寬-K 不可擬合」的說法是錯的，K 效應是可以擬合的（見 P2）。
- 沿 N 分塊、連續上升：當 N 超過一個 tile 寬（>2048）就切多塊，每塊按自己的大小計費——所以延遲是持續往上，不是前一版那種「跳到 2 倍再變平」。（一個只用一小段的第二塊，因為塞不滿、吞吐低，會帶一點開銷，所以 N 剛過 2048 時有一小跳，之後續升。）

單位更正：Metis 是 INT8 整數引擎 → 吞吐是 GOP/s（giga-OPS），不是 GFLOP/s。本章已全部改正（issue #18）。

A1.4 MEASUREMENT VS PREDICTION（誠實版）

先講清楚什麼有量、什麼沒量。decode 投影（proj_decode）共 16 個形狀，但只有 5 個是原生量測（N≤2048、K·N≤4.19M，能配置進裝置），其餘 11 個 N 或 K 太大、超過 envelope，根本沒量到——它們的「值」是我們公式 tile-sum 算出來的，不是量測。所以這一版不再把那 11 個列成「量測 vs 預測、誤差 0%」。

（A）原生量測的 proj_decode（5 個，這才是真的「量測 vs 預測」）：

模型	投影	K	N	量測 MS
1B	q_o	2048	2048	41.2
1B	kv	2048	512	18.6
3B	kv	3072	1024	34.5
8B	kv	4096	1024	36.9
Qwen	kv	3584	512	24.9

(B) 沒有量測、由公式 **tile-sum** 合成的 (11 個, 標明無量測) : 1B/3B/8B/Qwen 的 q_o (大)、 $gate_up$ 、 $down$ ——這些 $K \cdot N$ 從 9.4M 到 68M, 全部 $> 6M$ **envelope**, 沒辦法在裝置上量。它們的延遲是模型輸出 (165–824 μ s), 不是量測值, 不該當驗證。

真正的擬合驗證在「吞吐」上 (13 個原生單-tile 點) :

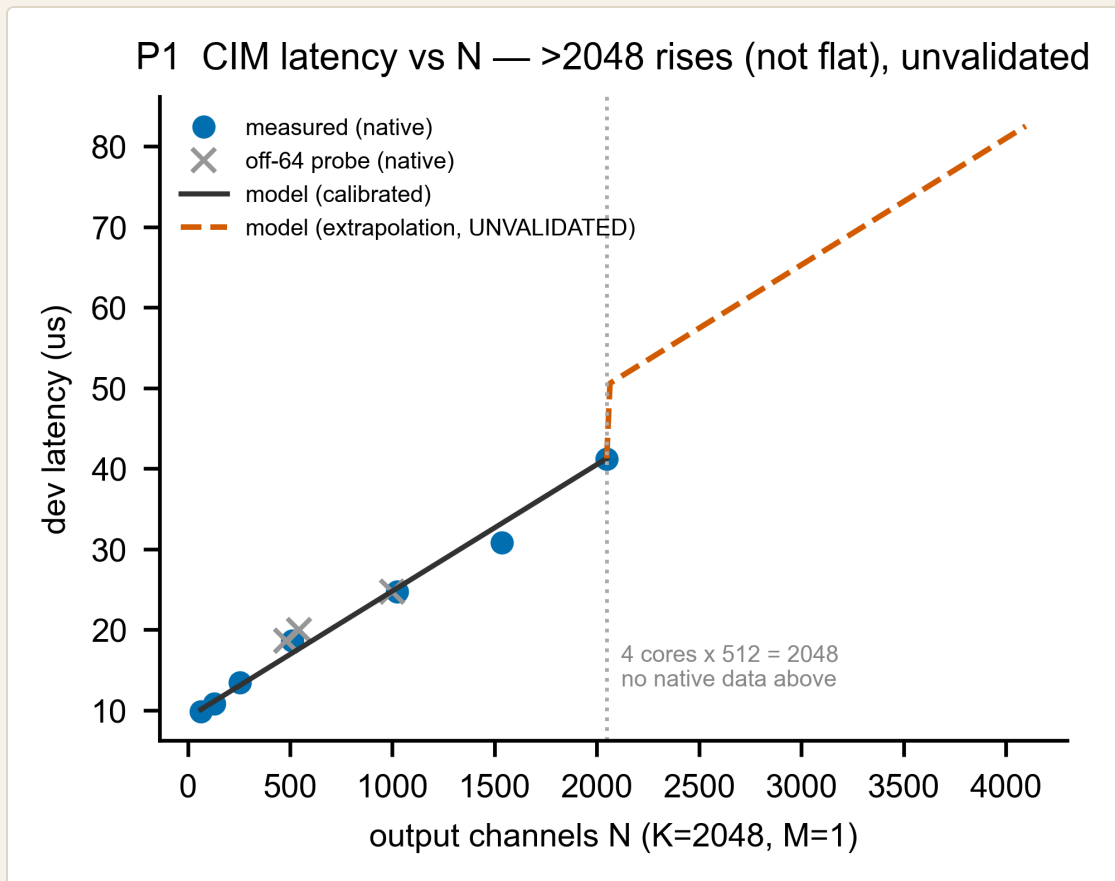
- **G_eff(N,K)** 擬合誤差 : 中位數 2.7%、p95 14.9%、max 17.6% $\rightarrow$ 過門檻 ($median \leq 10\%$ 、p95 $\leq 20\%$) 。
- 由吞吐導出的原生延遲誤差 : 中位 2.7%、max 21%。
- 多-tile 唯一的原生點 ($N=4096, K=1024 = 37.1\mu s$, 這是唯一 $N>2048$ 還能配置的原生點, 因為 $K=1024$ 小) : 模型預測 50.3 μs (+36%, 偏高)。代表我的連續分塊外推偏悲觀 (真實硬體在多塊時 pipeline 得更好), 這點誠實標為「外推、偏高、未充分驗證」。

A1.5 K 效應是可以擬合的 (更正前一版)

前一版說「寬-K 幫助吞吐只有 8B kv 一個點、不能擬合」——這是錯的。實際有 13 個原生點橫跨 N 與 K , 固定 N 就能看到 K 效應 (上表 : $N=512$ 時 $K_{2048} \rightarrow 112.6$ 、 $K_{3584} \rightarrow 147.4$; $N=1024$ 時 $K_{2048} \rightarrow 170$ 、 $K_{4096} \rightarrow 227$) , 而且 aspect 點顯示它對稱 ($N_{1024} \cdot K_{4096} = N_{4096} \cdot K_{1024} = 227$) 。我已改用 **2D G_eff(N,K)** 把 K 一起 fit 進去 (P2 是它的對比圖) 。殘留 : 高-K 角落 (如 8B kv 的 227) 公式仍略微低估 (P2 看得到那顆點在線上方) , 這個殘差據實呈現、不再藏。

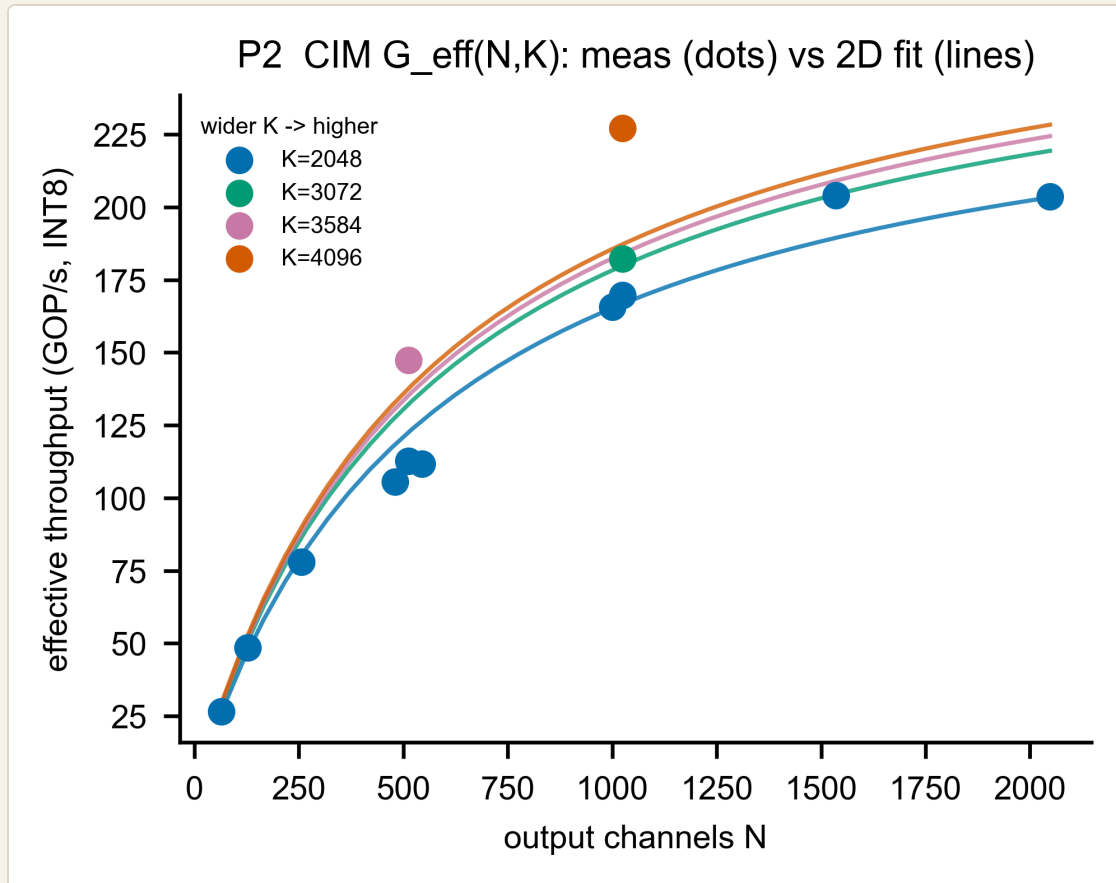
A1.6 圖片解釋

圖 A1-1 (P1) — 延遲 vs N (K=2048) : 原生 + 外推



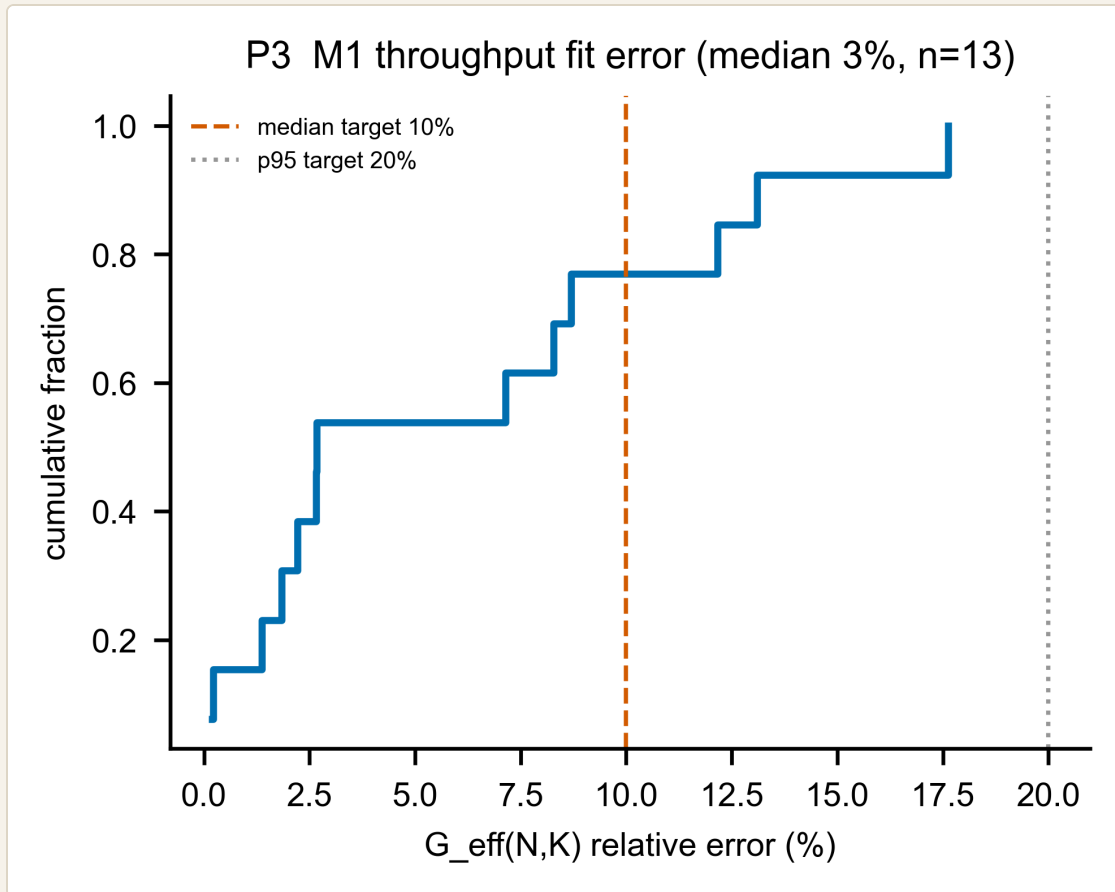
- X：輸出通道 N。Y：device 延遲 (μs)。
- 藍點 = 原生量測，灰 × = 非-64 探測點。黑實線 = 校準範圍 ($N \leq 2048$) 的模型。橘虛線 = $N > 2048$ 的外推 (UNVALIDATED)。
- 怎麼看： $N \leq 2048$ 模型貼合量測；過了 2048 (=4 核×512、無原生資料) 後虛線持續上升（不是前一版的平移），並明確標「未驗證」。前一版被你抓到的假 $N=3072$ 量測點已移除。

圖 A1-2 (P2) — 2D 有效吞吐 $G_{\text{eff}}(N,K)$: 量測 vs 擬合



- X：輸出通道 N。Y：有效吞吐（GOP/s，INT8）。顏色 = 不同 K。
- 點 = 量測，線 = 2D 擬合。
- 怎麼看：同一個 N，K 越大（顏色越暖）吞吐越高——這就是 K 效應，而且被公式抓到。橘色 K=4096 在 N=1024 那顆點（227）落在自己擬合線上方，誠實反映公式對高-K 角落略低估。

圖 A1-3 (P3) — **G_eff** 擬合誤差分佈



- X : **G_eff(N,K)** 相對誤差 (%)。Y : 累積比例。垂直線為 10%/20% 門檻。
- 怎麼看：曲線整體落在門檻左側（中位 2.7%），但有少數點接近 15–18%（高-K 角落），這些都在圖上、沒有被平均蓋掉。

A1.7 限制與 GAP (誠實清單)

項目	狀態	說明
單-tile 吞吐 <code>G_eff(N,K)</code>	✅ 已擬合驗證	13 個原生點，中位 2.7%、p95 14.9%
多-tile (N>2048 / K·N>4.19M)	❌ 未驗證	只有 1 個原生多-tile 點；模型偏高 +36%；連續上升外推、明標 unvalidated
proj_decode 大形狀 (11 個)	❌ 無量測	K·N>6M envelope，裝置量不到；值為模型 tile-sum，非量測
n_cores	⚙️ 參數化	最小單位=單核 512；n 可調；本次校準在 n=4 (cores-spanned 待上板確認)
單位	✅ 已更正	GOP/s (INT8)，非 GFLOP/s
compute ceiling	🚫 未建模	量到的 ~227 GOP/s 只有峰值 209,600 GOP/s 的 0.1%；decode 不碰天花板 (issue #16)
64 對齊	✅ 已解釋	64 outputs/cycle、8×64 輸出組織；非-64 點無明顯罰則
device envelope	⚠️ 證據到 IOMMU window	6M ≈ 預設 PCIe-IOMMU window (~14MB，forum #1330) 的權重份額 (Alpha 無真 on-card DRAM)；非 L2(32MB)/SRAM；確切 window 大小待上板看 <code>compile_config.json</code>
板子離線	🛑 無法補量	多-tile / >2048 / 大形狀都無法再量；只能外推並標未驗證

一句話總結 A1：Metis CIM 是四核、每核 512×512 的 INT8 D-IMC；我們以「n 核」為最小單位 (n=4) 建模，有效吞吐 `G_eff(N,K)` 隨 N 與 K 上升、在 13 個原生點上擬合到中位 2.7%；但只有單-tile (≤一塊) 有量測，更大的形狀全是外推、誠實標未驗證 (連續上升，不再平移，也不再把外推點當量測)。下一章 A2 處理「搬資料」那一半。

A2 — M2：記憶體階層（「搬資料」那一半）

這一章你會學到：為什麼「搬資料」跟「算」一樣重要、PCIe 上那筆神祕的 911μs 固定成本是什麼、它什麼時候要付什麼時候不用付、host 記憶體（LPDDR5）我們怎麼估、以及 KV-cache 這個在長文本下會吃掉 1/3 頻寬的隱形成本。

A2.1 架構考量：M2 是誰？為什麼「搬」這麼重要？

A1 的 M1 算的是「算」——CIM 在 crossbar 上做乘加。但資料不會自己出現在 crossbar 上，得搬進去、結果再搬出來。M2 就是負責「搬」這一半，對應 §0.7 架構的「④ 記憶體階層」。

為什麼搬這麼重要？回顧 §0.3：decode 是 memory-bound（卡在搬權重，不是卡在算）。所以一個 8B 模型每吐一個 token 要搬 7.5 GB 權重——搬得多快，幾乎就決定了 decode 多快。M1 算得再快，搬不過來也沒用。

M2 要模型化三件事：

子模組	負責什麼	對應
2a PCIe / DMA	host ↔ CIM 之間「一次搬移」要多久	那筆 911μs 固定成本 + 頻寬
2b host DRAM	host 主記憶體頻寬（量測=量產卡 LPDDR4x；模擬 SoC=LPDDR5）	量測 ~24 GB/s 的記憶體牆
2c kv_cache	每個 token 把新的 K/V 存進快取的成本	長文本下的隱形頻寬

A2.2 原理 + 參數（2A：PCIe 與那筆 911MS 固定成本）

先講 PCIe 和 DMA 是什麼。CIM 是一張插在 PCIe 插槽上的卡（就像顯卡）。host（主處理器）要餵資料給它，得透過 PCIe 做一次 DMA（Direct Memory Access，直接記憶體存取）——把一塊資料從 host 記憶體搬到裝置。每一次這樣的搬移，都有兩部分成本：

一次搬移時間 = 固定開銷 (floor) + 資料量 ÷ 頻寬

transfer_us = 911 μs + bytes ÷ 3.9 GB/s

- 固定開銷 (floor) = 911 μ s：不管你搬多少資料，光是「發起一次 DMA」這個動作就要約 911 微秒（建立傳輸、等待、同步...）。這是我們從真晶片量出來的常數。
- 頻寬 = 3.9 GB/s：PCIe Gen3 $\times 4$ 的文件規格值；資料量越大、這部分越久。

這筆 911 μ s 為什麼這麼關鍵？因為 decode 的單次計算很小（M1 算過，一個小 GEMV 的 device 時間只有 ~18–165 μ s），可是只要它得走一次 PCIe，就要先付 911 μ s——固定成本是計算本身的 5–50 倍！這就是 CIM「算得快、卻輸在來回」的結構性弱點。

⚠ 最重要的一個邊界：這筆 floor 什麼時候付、什麼時候不付？

這裡要很小心，因為它牽涉到「我們量的板子」和「我們模擬的目標」是兩回事（§0.4 提過）：

- 我們量測的 Alpha 板：沒有 on-card DRAM（卡上沒有自己的記憶體），所以連權重都得每次從 host 走 PCIe 搬 → 每次 decode 呼叫都付 911 μ s。這是該板的拓樸限制，不是 CIM 的本質。
- 我們模擬/預測的目標（量產卡那種有 on-card DRAM 的拓樸）：decode 串流權重時，權重就在卡上的 DRAM，不需要每次走 PCIe → decode 不每次付這筆 floor。

所以模型把 floor 的「適用範圍」講清楚：floor 只對「離散的 host $\leftrightarrow$ device 搬移」收費——也就是 KV-reload、activation handoff、混合精度的 conversion-op 這類真的要過 PCIe 的流量；而 decode 串流權重的主幹用頻寬項、不逐次付 floor。Part B 的端到端預測就是照這個邊界，故意不把 Alpha 的 911 μ s 外推到量產卡。

參數怎麼來的？我們沒有做「不同資料量 $\times$ 傳輸時間」的完整掃描（那是 Phase 0.3 的一個 collect-what-you-can 缺口），所以 911 μ s 和 3.9 GB/s 都當固定參數，不重新擬合斜率。這點在報告裡誠實標註。

補充：什麼是 double buffering（雙緩衝）？一次推論其實有三部：搬輸入進去 → 裝置計算 → 搬輸出回來。

- 關閉（單緩衝）：三步串列——搬完才算、算完才搬回，DMA 和計算互相等待，總時間 $\approx$ 搬入 + 計算 + 搬回。
- 開啟（雙緩衝）：用兩塊 buffer 輪流，讓「這一筆在算」的同時「下一筆已經在搬」——DMA 和計算重疊（pipeline），總時間 $\approx$ max(搬, 算)，把搬移的時間藏進計算裡。

對 CIM 的意義：decode 的計算很小（A1 那 9–165 μ s），所以開雙緩衝能藏掉的有限，911 μ s 的 per-call floor 仍主導。Phase 0.3 量 PCIe 時就是用 `--double-buffer` / `--no-double-buffer` 兩組對比去隔離出傳輸成本（voyager-sdk §A2）。這也呼應 A1.5 那條 $N > 2048$ 外推：大運算時雙緩衝能藏更多搬移，是「真實硬體比我悲觀的連續分塊更好」的原因之一。

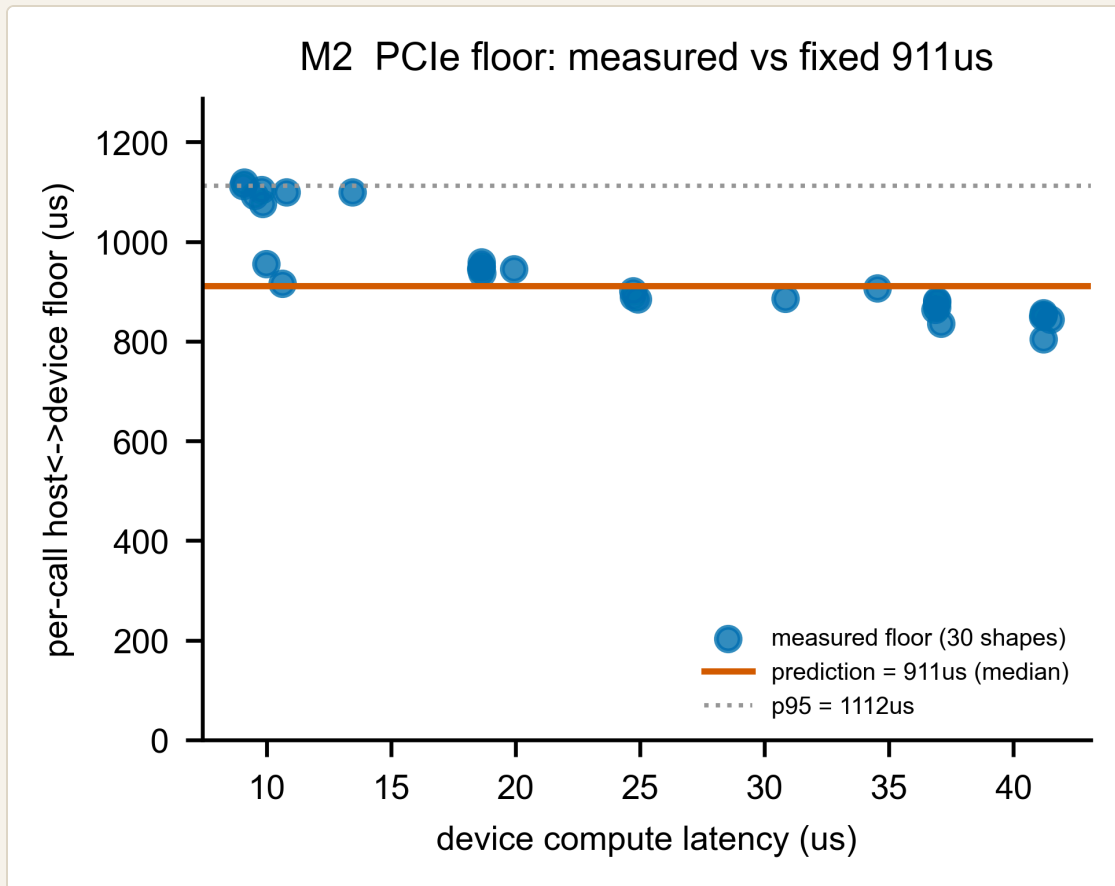
A2.3 MEASUREMENT VS PREDICTION (2A：那筆 FLOOR 真的是固定的嗎？)

我們怎麼量到 911 μ s 的？很巧妙：同一個運算量兩個值——

- **device latency**：只算「在裝置上計算」的時間（M1 那個）。
- **system latency**：從 host 角度看的「整趟來回」時間。

兩者相減 = 那一趟付的固定來回成本（**floor**）。我們在 30 個單-tile 形狀上各算一次，得到下圖：

圖 A2-1（M2 PCIe floor）— 量測的 floor vs 固定 911 μ s 預測



- X 軸：device 計算延遲（ μ s）——代表「這個運算多大」（這張圖只取單-tile 形狀，範圍約 9 μ s 到 41 μ s）。
- Y 軸：實測的每次呼叫 floor（= system - device， μ s）。
- 藍點：30 個形狀各自量到的 floor。橘線：我們的預測（911 μ s 中位數）。灰虛線：p95（1112 μ s）。
- 怎麼看（關鍵）：藍點全都落在一條 805–1120 μ s 的帶裡——和它對比的「資料量項 **bytes/BW**」對這些小 decode GEMV 幾乎是 0（搬幾 KB $\div$ 3.9GB/s $\approx$ 零點幾 μ s）。所以這趟來回的成本幾乎全部來自那筆固定開銷，是不折不扣的主導項。
- 但要誠實看圖：藍點不是完全水平——你會看到它隨運算變大微微往下滑（小運算約 1100 μ s $\rightarrow$ 大運算約 810 μ s，統計上負相關 $r \approx -0.86$ ，約 -7 μ s/ μ s）。這可能是大運算時 DMA 和計算有部分重疊

(double-buffering) 所致。重點是：這個漂移幅度 ($\sim 300\mu\text{s}$) 遠小於 floor 本身 ($\sim 900\mu\text{s}$)，所以我們用一個固定常數 $911\mu\text{s}$ 當一階近似，並把這個與大小相關的漂移誠實地併入殘差帶(中位 911、p95 1112)。

換句話說：measurement = 沿著一條微降趨勢散在 805–1120 的藍點；prediction = 一條 $911\mu\text{s}$ 的水平線。模型抓對了「來回成本主導、約 $900\mu\text{s}$ 」這個本質，但沒有宣稱它跟運算大小完全無關——那個微降趨勢是已標註的殘差。

A2.4 LPDDR5 HOST 記憶體 (2B：解析模型，誠實標註)

模擬的 SoC 用 LPDDR5 (行動裝置常見的低功耗 DRAM) 當 host 主記憶體。我們需要知道它的有效頻寬 (每秒能串多少權重)。

- 那個 24.2 GB/s 是量測的——而且量的是量產卡的 LPDDR4x (不是 LPDDR5)。來源：量產 Metis Card 的 batch-1 decode 是純權重串流的記憶體牆 (decode 時間 $\propto$ 權重 bytes, $r^2=0.997$)，反推有效頻寬 ≈ 24.2 GB/s (voyager-sdk.md:278)。
- 為什麼只有「理論的一半多」？因為它本來就該如此——而且先前我比錯了記憶體。量產卡用 LPDDR4x，不是 LPDDR5：
- LPDDR4x-4266 $\times 64$ -bit 理論峰值 ≈ 34 GB/s $\rightarrow 24.2/34 = 71\%$ ，是 memory-bound decode 的正常效率。
- 文獻佐證：HeteroInfer (SOSP'25, 本 repo, Fig 5) 量到 Snapdragon 8 Gen 3 的單一處理器 decode 只達 40–45 / 68 GB/s = 59–66%；網路綜述也是 60–80% (未優化可低到 21%、最佳 FPGA 達 85%)。所以 $\sim 71\%$ 完全在這個帶內。
- 我先前寫「47% of LPDDR5-51.2」是錯的——拿量測的 LPDDR4x 去除以 LPDDR5 的峰值 (不同記憶體) 才會看起來異常低。已更正成 71% (對 LPDDR4x)。
- 模擬的前瞻 SoC 才用 LPDDR5 (橋接假設)：峰值 51.2 GB/s (LPDDR5-6400 $4 \times 16b$)，有效 $\approx 51.2 \times 0.65 \approx 33$ GB/s。這是另一塊記憶體、另一個數字，別跟量測的 24.2 混為一談。

⚠ 誠實標註：DRAM 模型是解析的 (Ramulator2 後端：單串流 LPDDR5 cross-check 在 Phase 1.3、多單元競爭在 Phase 2, ADR-0002)；量產卡 LPDDR4x 的確切 bus 寬度未公開 ([GAP])。驗證 = 合理性檢查：量測有效頻寬落在峰值的 55–85% (71% ✓，且對齊上述文獻)。

A2.5 KV-CACHE (2C：長文本下的隱形成本)

KV-cache 是什麼？decode 時，模型要「回看」前面所有 token 的 Key/Value。為了不每次重算，把每個 token 的 K、V 存起來 (cache)，下個 token 直接讀。每生一個新 token,就要把它的 K/V **append** (附加) 進 cache——這是一個純記憶體頻寬的動作 (搬 bytes,不做計算)。

我們的模型很簡單： $\text{kv_append_us} = \text{kv_bytes} \div \text{有效頻寬}$ 。

為什麼要特別講它？因為在長文本下它一點都不小：Phase 0.2 的統計顯示，在 LongBench(prompt 約 11800 個 token) 的 decode 階段，kv_cache 佔了 12.6-33.5% 的 bytes (8B 模型 22.2%、3B 最高 33.5%) ——在多數模型是僅次於 matmul、attention 的第三大頻寬消耗(而 3B 模型甚至超過 attention、躍居第二)。短文本可忽略，長文本不行；Phase 2 要跑 LongBench,漏了它會系統性低估。

⚠ 誠實標註 + 暫時做法：Phase 0.3 沒有單獨量測 KV-append (collect-what-you-can 缺口),而且板子現在離線、補不了量。所以現階段就用這個解析式 $\text{kv_bytes}/\text{有效頻寬}$ 當暫時替身(用已有的頻寬數字)、明標 **unvalidated**——這完全夠 Phase 2 先把模擬器跑起來；等板子救回再用真值校準 (它是純頻寬 op,解析式的形式是對的，只差一個係數確認)。比照 A1 對「lm_head / prefill」的誠實處理。

A2.6 一個更新的設計決定：PHASE 1.2 要建 L1/L2 RESIDENCY

這條決定改了。前一版這裡寫「不要建 SRAM residency」，理由是 Phase 0.3 量到 Alpha 的 l2-vs-ddr 延遲比 1.00-1.01× (幾乎沒差)。但這個結論只在 Alpha 上、而且只因為 Alpha 根本沒有 on-card DRAM (所謂「ddr」是 host LPDDR over PCIe),所以「放 L2 還是放 DRAM」在那塊板上不是有意義的軸——它是拓樸特例，不能外推成「不必建」。

更新 (依架構研究需求):Phase 1.2 要建 L1/L2 residency 模型。因為這個模擬器之後要做架構研究：一旦我們把當前的「記憶體牆」瓶頸解掉 (換更快的記憶體、加大 SRAM、改拓樸...),L1 (每核 4 MiB) /L2(32 MiB 共享) 能不能放下權重/KV,就會變成決定性的變數。要回答「如果權重能整個放進 L2,decode 會快多少」這類問題，就必須把真實的 SRAM 階層還原進模擬器。所以：

- 建 L1 (4 MiB/核) + L2(32 MiB 共享) 的 residency 模型 (尺寸見 A1.2 的 device-envelope 段與 [AIPU 論文筆記](#));資料放 L1/L2/DRAM 哪一層、各層頻寬與容量，都要顯式建模。
- Alpha 的「l2/ddr 無差異」量測只描述 Alpha 這塊板，不能當「不必建」的依據。
- M2 合約已從「不建」改成「Phase 1.2 建 L1/L2」。
- 一切硬體都還原最真實的情況——這是讓模擬器能拿來做架構探索的前提。

A2.7 限制與 GAP（誠實清單）

項目	狀態	說明
PCIe floor / BW	✅ 固定參數	911μs + 3.9 GB/s;無逐點 PCIe 掃描（Phase 0.3 缺口）→ 不重擬斜率
DRAM 頻寬	⚠️ 解析	量測 = LPDDR4x 24.2 GB/s = 71% of ~34 peak（HeteroInfer 59–66% + web 60–80% 佐證）;模擬 SoC 的 LPDDR5 另計（33/51.2）;Ramulator2 單串流在 Phase 1.3、多單元在 Phase 2
kv_cache append	❌ 未驗證（暫用）	解析 kv_bytes/BW 當暫時替身（板離線）;長文本佔 12.6–33.5% 不可省；待救板補真值
L1/L2 residency	🔧 改為要建	Phase 1.2 建 L1（4MiB/核）+L2（32MiB 共享）;Alpha「l2/ddr≈1.0」只是無 on-card DRAM 的拓樸特例，不能外推
floor 適用邊界	✅ 已界定	只對離散 host↔device 搬移收費；decode 串流不逐次付（見 A2.2）

一句話總結 A2：「搬資料」的成本由「一次約 911μs 的固定來回 + 頻寬」描述，這筆來回成本主導（對小 GEMV 而言 bytes/BW 幾乎是 0）;記憶體頻寬方面，量測到的 24.2 GB/s 是量產卡 LPDDR4x 的有效值（≈71% 理論峰值，HeteroInfer 與網路文獻都佐證這是正常），模擬 SoC 的 LPDDR5 另算；KV-cache 用解析式當暫時替身（板離線、未驗證）;L1/L2 residency 改為 Phase 1.2 要建(架構研究需要還原真實 SRAM 階層)。M1（算）+ M2(搬) 合起來，就是 CIM 路徑的完整成本——接下來 A3/A4 看 CIM 不擅長的事丟給誰做。

A3 — M4-GPU：Mali GPU（attention 的接收方）

這一章你會學到：為什麼 attention 一定要從 CIM「外包」出去、GPU 怎麼接這個球、它的成本公式長怎樣（而且超準，誤差 0.6%）、以及一個誠實但重要的事實——這顆 GPU 的矩陣乘法其實很慢，慢到我們只能把它的絕對值當「下界」用。

A3.1 架構考量：GPU 在系統裡是誰？

回顧 §0.4 的 CIM-centric 哲學：CIM 擅長 weight-stationary 的矩陣乘法，但不擅長 attention。那 attention 丟給誰？本期我們用 Mali-G610 GPU 當 attention 的 offload（外包）接收方。

在 §0.7 的架構裡，GPU 屬於「③ 各單元時間模型」的一格，是 CIM 的支援層。它要回答的問題：給一段 attention（KV 長度 = kv），GPU 算它要多久？

為什麼是 GPU 不是 NPU？兩個都是候選。但本期 NPU（RKNPU2）卡在 issue #13 沒量到（見 A5），所以這一輪的 offload 對照組是 GPU。「attention 該 offload」這個結論，目前站在 GPU 的數據上。

A3.2 原理：為什麼 ATTENTION 對 CIM 是毒藥、對 GPU 是日常？

回顧 §0.3，attention 的核心是兩個 batch 矩陣乘法（bmm，即「每個 head 各做一次的小矩陣乘法」，一批 head 一起算所以叫 batch）：

1. QK^T ：把 query 和所有 key 做內積（算「這個 token 該注意誰」）。
2. $S \cdot V$ ：把注意力權重和所有 value 相乘（把該注意的資訊加權取出）。

關鍵差異：這兩個 bmm 的兩個運算元都是「會隨 token 變動的中間資料」（這個資料在 ML 裡叫 activation／激活值，注意不是 §0.3 那個激活函數 SwiGLU）——K 和 V 是隨著 token 一直長大的 KV-cache，不是固定的權重。

- 對 CIM：weight-stationary 的前提是「有一個固定的權重釘在 crossbar 上」。但 attention 沒有固定權重——每個 decode step 都得把長大的 K/V 重新載入 crossbar。這就違反了 CIM 省電的本質（A1 講過），成本爆炸（Part B 會算出 CIM attention 要 31–46 ms/token）。

- 對 GPU：GPU 本來就是設計來算「activation × activation」的通用矩陣乘法，這對它是日常。所以 attention 在 GPU 上只要幾十到幾百微秒。

這就是「attention 要 offload」的硬道理：不是偏好，是 CIM 的物理限制。

澄清（你問的 Q6）：attention 也包含矩陣乘法，那「GEMV 適合 CIM、attention 適合 GPU」到底在分什麼？分的不是「有沒有矩陣乘法」——投影（Q/K/V/O、FFN）和 attention（ QK^T 、 $S \cdot V$ ）全都是矩陣乘法（decode 時都是 GEMV/bmm）。真正的分界是「其中一個運算元是不是固定的權重」：

- 投影 / FFN：activation × 固定模型權重（ W_q 、 W_{gate} ... 是訓練好的常數）→ 權重可以釘在 crossbar 上不動（weight-stationary）→ CIM 的主場。
- attention：activation × activation（Q、K、V 全是 runtime 資料，K/V 還隨上下文長大）→ 沒有固定權重可釘，每個 decode step 都要把長大的 K/V 重載進陣列 → 違反 CIM 省電前提 → 丟給 GPU/NPU（它們做通用 activation×activation 是日常）。

所以精確的講法是：weight-stationary 的 matmul → CIM；activation×activation 的 matmul → GPU/NPU。我先前「GEMV 適合 CIM、attention 適合 GPU」是簡化說法，這裡是精確版。

A3.3 參數設計：ATTENTION 成本公式

我們量了 GPU 上單一 attention head 的 $QK^T + S \cdot V$ ，在三個 KV 長度（ $kv \approx 128 / 512 / 1024$ ），用 FP16。發現成本隨 kv 長度線性增加——很合理，因為 kv 越長、要算的內積越多。於是擬合一條直線：

$$\begin{aligned} \text{attn_bmm_us}(kv) &= a + b \cdot kv \\ &= 27.74 + 0.442 \cdot kv \quad (\text{單一 head, FP16, } \mu\text{s}) \end{aligned}$$

- $a = 27.74 \mu\text{s}$ ：截距，可理解為「啟動一次 attention 的固定開銷」。
- $b = 0.442 \mu\text{s}/kv$ ：斜率，每多一個 KV token 多花 $0.442 \mu\text{s}$ 。

單一 head 的注意事項：這個公式是一個 head 的成本。真正一個 token 的 decode attention 要乘上「head 數 × layer 數」。這個 **× heads × layers** 的聚合方式，我們留到 Phase 2 才展開（這是一個已記錄的 watch-item；因為在短文本下 attention 遠小於權重串流，不影響 decode 的主要 gate）。

⚠️ 但 kv 只量到 1024，我們的 workload 最長到 ~12k（你問的 Q5）。LongBench 的 prefill ≈ 11.8k token（Qwen 12.2k）→ decode 時 kv 會長到 ~12k，是量測上限 1024 的約 12 倍。

- 線性形式本身是對的：decode 的 QK^T （ $[1,hd] \times [hd,kv]$ ）和 $S \cdot V$ （ $[1,kv] \times [kv,hd]$ ）都是對 kv 個元素各做一次（ $O(kv)$ ），softmax 也是 → per-step 成本結構上就線性於 kv。所以 $a + b \cdot kv$ 的形狀是正確的、不是硬湊。
- 但把它外推 12× 是未驗證的：斜率/截距是在 $kv \leq 1024$ 擬的，到 12k 可能因 cache 行為、GPU 占用率而偏移。連 HeteroInfer 的特性化也大多在 $seq \leq 1024$ （其 Fig 8 最長序列就是 1024）。所以 $kv \gg 1024$ 標為「未驗證外推」，救板後應在 $kv \in \{2k, 4k, 8k\}$ 補點確認線性是否延續。這也是 §A1/Part B 那條「prefill 路徑未驗證」之外、decode 端的一個明確 gap。

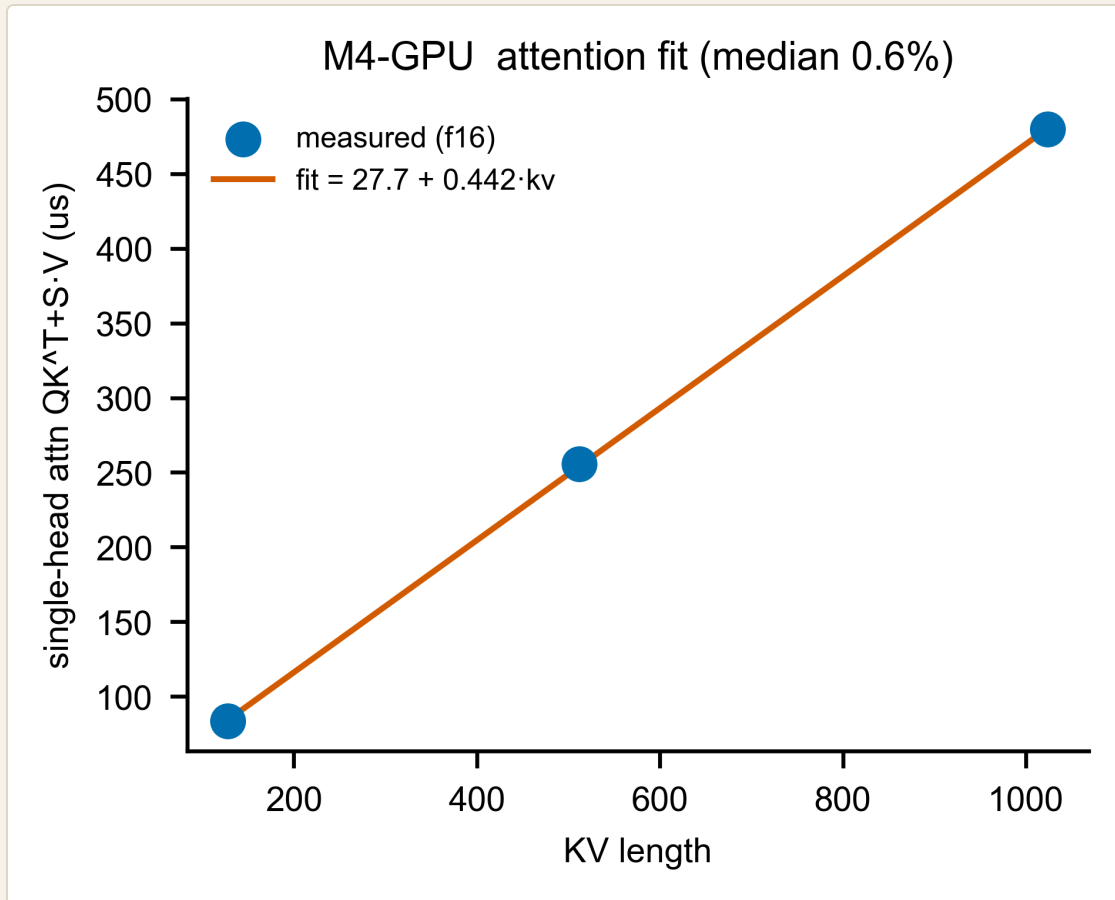
A3.4 MEASUREMENT VS PREDICTION (ATTENTION 公式準不準?)

直接比量測點和公式線：

KV 長度	量測 MS	公式 MS	相對誤差
128	83.4	84.3	+1.1%
512	255.7	254.1	-0.6%
1024	479.7	480.4	+0.1%

相對誤差中位數 0.6%、p95 1.1%——遠遠優於門檻（10%/20%）。這是全報告擬合最漂亮的一個（因為 attention 對 GPU 是「線性、規律」的工作）。

圖 A3-1 (M4-GPU attn fit) — attention 量測 vs 公式



- X 軸：KV 長度。Y 軸：單一 head 的 $QK^T + S \cdot V$ 延遲 (μs)。
- 藍點：真晶片量測。橘線：擬合的直線 $27.74 + 0.442 \cdot kv$ 。
- 怎麼看：三個藍點幾乎完美落在橘線上——線性關係成立、公式抓得極準。這就是「規律的工作容易模型化」的最佳例子（對照 A1 的 CIM 有 GQA underfill 的不規律殘差）。

A3.5 一個誠實的大坑：這顆 GPU 的矩陣乘法其實很慢

這裡要講一件容易誤會的事。除了 attention，我們也量了 GPU 做矩陣乘法（投影那種）。結果很慘：

- **ksweep**（方陣掃描）：GPU 的矩陣乘法吞吐到 $M=128$ 就飽和在約 20 GFLOP/s——這對一顆 GPU 來說很低。
- **decode GEMV** ($M=1$) 更慘：例如 1B 的 q_{lo} 投影，GPU 量到約 19.5 毫秒 (ms!)，而同樣的運算 CIM 只要 41 μs ——GPU 慢了約 470 倍。

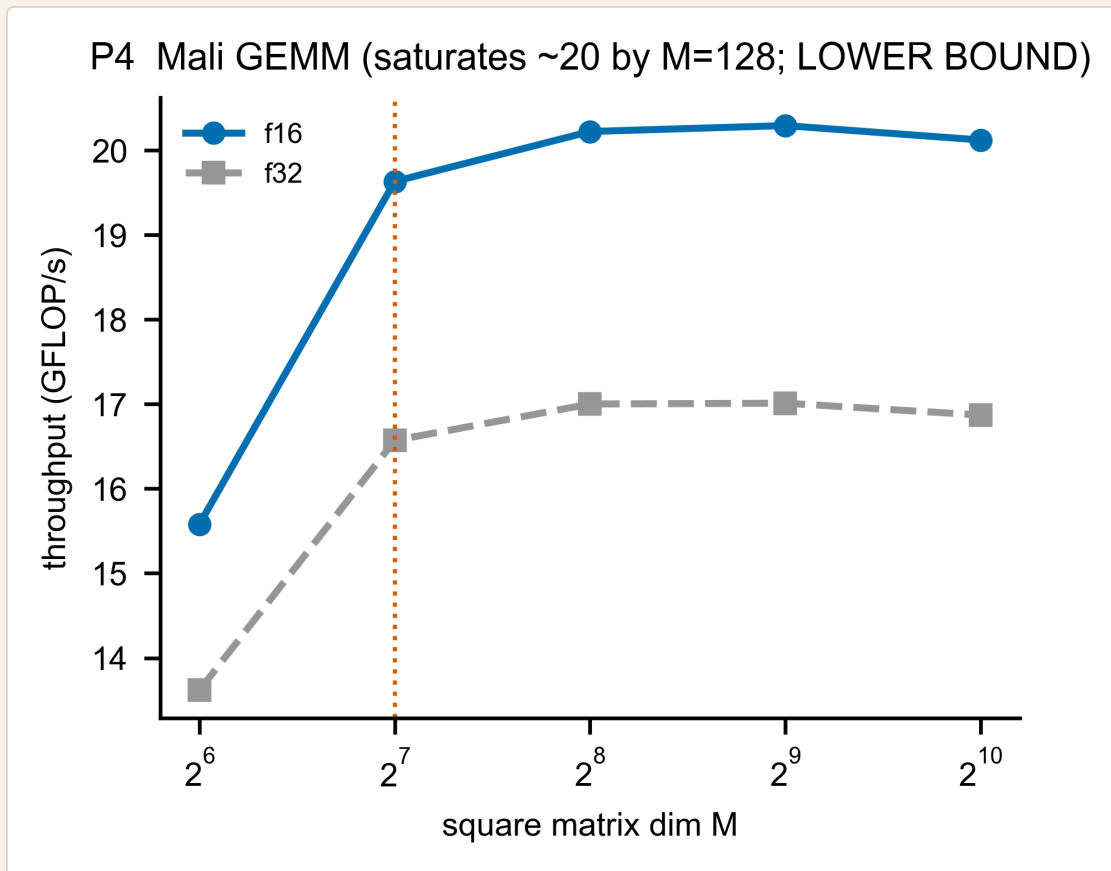
為什麼這麼慢？兩個原因：

1. 我們的 OpenCL kernel 是自己寫的、沒優化（為了避開框架雜訊）。一顆調教過的 Mali kernel 會快很多。
2. decode 是 $M=1$ 的 GEMV，GPU 利用率極低（GPU 喜歡大批次平行；一次只算一個 token 會嚴重 under-utilise，量到的吞吐只有約 1.3 GFLOP/s，遠低於飽和的 20）。

所以我們怎麼誠實處理？

- GPU 的矩陣乘法絕對吞吐標為「下界（lower bound）」——意思是「真實的 Mali 至少有這麼快，可能更快」。我們只取它的「形狀趨勢」，不押它的絕對值。
- 這也正好解釋了為什麼矩陣乘法要交給 CIM 而不是 GPU：GPU 算 decode 投影又慢又耗電，CIM 才是對的單元。GPU 的價值在它能做 CIM 做不了的 attention。
- 好消息：A3.4 的 attention 對照（CIM 31–46ms vs GPU 幾十–幾百 μ s，差約 2 個數量級）對 kernel 品質不敏感——就算 GPU kernel 沒優化、慢個幾倍，CIM 還是慢它兩個數量級。所以「attention 該 offload」的結論穩固。

圖 A3-2 (P4) — GPU 矩陣乘法吞吐 vs 大小



- X 軸：方陣維度 M （對數）。Y 軸：吞吐（GFLOP/s）。藍=FP16、灰=FP32。
- 怎麼看：吞吐在 $M=128$ 就大致持平在 ~20 GFLOP/s（之後僅在 20 附近微幅起伏，不再明顯上升；標題寫「LOWER BOUND」提醒這是未優化 kernel）。FP16 全程 $\geq$ FP32（半精度較快）。

對照 HeteroInfer（你問的 Q5：GPU 差距 + Fig 1 特性）。

- 差距 ~50×：HeteroInfer（SOSP'25，本 repo）在旗艦 Snapdragon 8 Gen 3 的 Adreno 750（優化 OpenCL kernel）量到 GPU matmul 約 1 TFLOPS（理論峰值 2.8 TFLOPS）；我們的 Mali-G610（RK3588，中階）+ 未優化 kernel 飽和在 ~20 GFLOP/s → 約 50× 低。這個差距完全合理：旗艦 vs 中階 GPU、優化 vs 未優化 kernel——正好印證我們把 GPU 絕對 matmul 吞吐當「下界」是對的。
- Fig 1 特性我們也有：HeteroInfer Fig 1 的 GPU 特性是「tensor 小時 memory-bound、變大後 FLOPS 線性上升、再大則 compute-bound 飽和」。我們的 P4（ksweep 到 M=128 飽和）就是同一條轉折——特性對得上，只是絕對值因 GPU 等級 + kernel 低約 50×。所以 attention 對照（CIM 31–46ms vs GPU 幾十–幾百μs，2 個數量級）即使把我們的 GPU 往上修 50× 仍然成立（Part B 的 offload 結論穩固）。

A3.6 限制與 GAP（誠實清單）

項目	狀態	說明
attention bmm 公式	✅ 已驗證	$27.74 + 0.442 \cdot kv$ ，誤差中位 0.6%——GPU 的核心交付物
GPU 矩陣乘法絕對值	⚠️ 下界	kernel 未優化；只取形狀趨勢，不押絕對值
heads × layers 聚合	🔴 Phase 2	單一 head 公式；聚合方式是 watch-item（短文本影響小）
NPU 作為另一 offload	❌ 未量測	卡 issue #13（見 A5）；本輪 offload 只有 GPU 對照

一句話總結 A3：GPU 接走 CIM 不擅長的 attention,它的成本是一條漂亮的線性公式（誤差 0.6%);至於矩陣乘法，這顆 GPU(在我們未優化的 kernel 下）又慢又耗電，正好反證「matmul 該留給 CIM」——但即使如此，CIM-vs-GPU 的 attention 差距（2 個數量級）依然穩固支持 offload。下一章 A4 看更零碎的輔助運算交給 CPU。

A4 — M4-CPU：ARM A76 CPU（零碎輔助運算的雜工）

這一章你會學到：除了矩陣乘法和 attention，LLM 還有一堆「零碎的小運算」要做，這些丟給 CPU；它們大多是固定成本（直接存量測值），只有 softmax 隨上下文長度變；以及一個關鍵的誠實標註——我們量的是 numpy 模擬的 FP16，所以這些數字是「上界」。

A4.1 架構考量：CPU 在系統裡是誰？

A1 的 CIM 做大矩陣乘法、A3 的 GPU 做 attention。但一個 LLM token 還有一堆既不是大 matmul、也不是 attention 的小運算：

- RMSNorm（正規化，讓數值穩定）
- RoPE（旋轉位置編碼，告訴模型 token 的位置）
- SwiGLU（FFN 的激活函數，§0.3 提的 `ffn` 那一類）
- residual（殘差相加）
- softmax（把 attention 分數變成機率分佈）
- sampling（從機率分佈挑出下一個 token，例如 argmax）

這些零碎、逐元素或小型 reduction 的運算，丟給 ARM A76 CPU（RK3588 的大核，cores 4-7）。在 §0.7 架構裡 CPU 是「③ 各單元時間模型」的一格，是最末端的支援層。

M4-CPU 要回答：給一個輔助 op（和它的尺寸），CPU 算它要多久？

A4.2 原理：這些 OP 為什麼「大多是固定成本」？

關鍵觀察：對一個固定的模型，這些 op 每個 token 的工作量幾乎是固定的：

- RMSNorm / RoPE / SwiGLU / residual：它們的尺寸由 hidden size（模型寬度）決定，而 hidden size 對一個模型是固定的。所以每次跑都差不多久 → 一個常數就描述得了。
- softmax 是唯一例外：它要對「目前所有 KV」算機率，KV 越長、要算越多 → 成本隨 kv 線性增加（和 A3 的 attention 同理）。

所以 M4-CPU 的策略很簡單：softmax 用一條公式，其他用「存下量測值」。

A4.3 參數設計

(1) softmax——唯一的真正「公式」：

```
softmax_us(kv) = a + b · kv          (per 模型、per 精度)
例 (8B、FP16) := 12.34 + 1.553 · kv → kv=1024 時約 1603 μs
```

我們在 kv = 128 / 512 / 1024 三個點量測，擬合出每個（模型，精度）的 a、b。

(2) 其他 5 個 op——「常數查表」，不是擬合：這點要講清楚：rmsnorm / rope / residual / swiglu / sampling 沒有做尺寸掃描（每個只量了一個代表性大小），所以它們不是擬合出來的公式，而是直接把量到的中位數存起來（一種 lookup）。例如 8B、FP16：

OP	量測值 (MS)
residual	41.7
rope_apply	144.4
rmsnorm	157.2
swiglu	557.8
sampling_argmax	985.5

跨模型的「 ∞ hidden」只是觀察、不是定律：我們確實看到大模型這些 op 較慢（hidden 較大），但因為每個 op 沒有 within-op 的尺寸掃描，我們不宣稱一條 成本 $\propto$ hidden 的擬合律，只把它當觀察記錄。

(3) 兩個重要的誠實標註：

- 1. 用量測值，不是用 FLOPs 算（對應 issue #10）：這些非-GEMM op 的「理論 FLOPs」是個很粗的下界（1 flop/元素），跟實際延遲差很遠。所以我們堅持用實測延遲，不用解析 FLOPs。
- 2. FP16 是 numpy 模擬的 → 上界（upper bound）：我們的量測腳本用 numpy 跑 FP16，但 A76 沒有原生快速的 FP16 numpy 路徑，numpy 用軟體模擬 → 慢很多。看數據就知道：

OP (8B)	FP16 MS	FP32 MS	FP16/FP32
rmsnorm	157.2	23.6	6.7×
sampling_argmax	985.5	52.5	18.8×

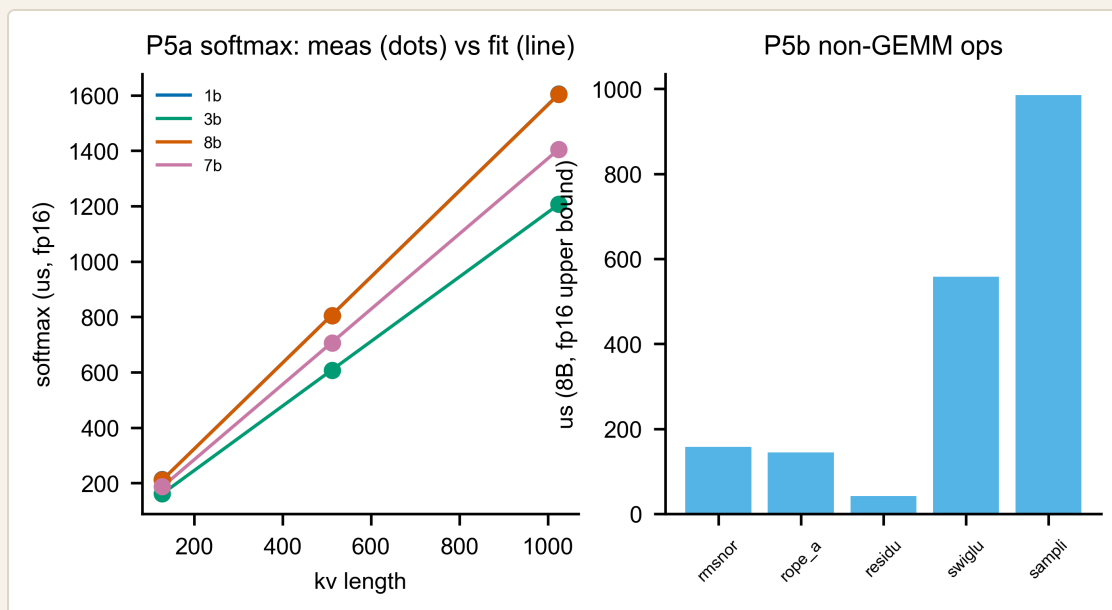
FP16 慢 FP32 達 7–19 倍，這不合理（半精度應該更快或差不多）——所以這是模擬造成的、是上界。真實硬體的 FP16 會更快。我們把 FP16 當「最壞情況」用，並明確標註 provenance(來源是 Phase 0.3 的收集註記，不是 cpu_ops.json 裡的欄位)。

A4.4 MEASUREMENT VS PREDICTION

softmax（唯一的擬合）：每個（模型，精度）用 3 個 kv 點擬合一條直線，共 8 條擬合（4 模型 × 2 精度），每條 3 個點 → 24 個殘差點（gate 的 n=24）；這些點對各自直線的相對誤差中位數 0.3%、p95 1.8%、max 3.4%，輕鬆過門檻（因為「3 點擬一線」幾乎完美）。

其他 5 個 op：它們的「預測」就是「量測值本身」（常數查表），所以談不上擬合誤差——這是誠實的：我們沒有為它們建模型，只是把真值存起來，等 Phase 1.2 建完整 CPU 模型、真要用某個沒量過的尺寸時再回頭量。

圖 A4-1(P5)— CPU 輔助運算



- 左圖 (P5a)softmax: 量測 vs 公式。X 軸：KV 長度。Y 軸：softmax 延遲 (us,FP16)。圓點 = 量測，線 = 擬合，每個模型一色。三點落在線上 → 線性成立、擬合準。

- 右圖 (P5b) 非-GEMM 常數。X 軸：op 名稱。Y 軸：延遲 (μs, 8B、FP16 上界)。長條 = 各 op 的量測常數。可看出 sampling_argmax 最貴 (~986μs)、residual 最便宜 (~42μs)。

A4.5 限制與 GAP (誠實清單)

項目	狀態	說明
softmax 公式	✅ 已擬合	$a + b \cdot kv$, 誤差中位 0.3%
其他 5 個 op	⚠️ 常數查表	無尺寸掃描；存量測值，非擬合
FP16 數值	⚠️ 上界	numpy 模擬，慢 7–19x; 真硬體會更快；明標 provenance
跨模型 ∞hidden	🔍 僅觀察	不宜稱擬合律
prefill 擴展	❌ 未驗證	這些都是 decode(1 token) 成本。prefill: 固定 op(rmsnorm/rope/residual/swiglu) 約 $\times S$; 但 softmax 因 per-token 本身就隨 kv 成長，對 S 個 token 加總 → 約 $S \times S$ (二次)。皆為解析外推、未驗證

一句話總結 A4：LLM 的零碎輔助運算交給 CPU, 其中只有 softmax 隨上下文長度變 (用線性公式，誤差 0.3%), 其餘是固定成本 (直接存量測值); 所有 FP16 數字因 numpy 模擬而偏慢，當上界用。到這裡，CIM (算 matmul) + GPU (算 attention) + CPU (算雜項) + 記憶體，四個計算路徑的成本模型都齊了——接下來 A5 補上缺席的 NPU, A6 看「是誰決定每個 token 要跑哪些 op」。

A5 — M4-NPU : RKNPU2（缺席的第二個 offload 候選）

這一章很短，而且故意誠實：這個元件本期沒有資料。它不是被遺忘，而是被一個外部障礙卡住（量測板離線）。我們把「為什麼沒有」「準備好了什麼」「對結論有什麼影響」講清楚——這正是「不藏 gap」的示範。

A5.1 架構考量：NPU 本來該是誰？

§0.4 講過：CIM 不擅長 attention，要 offload。A3 用 GPU（Mali）當 offload 接收方。但其實還有第二個候選——NPU（RKNPU2，RK3588 內建的神經網路加速器）。

NPU 和 GPU 一樣能做「activation × activation」的運算，所以也能接 attention；而且 NPU 一般預期比 GPU 更省電（這是 NPU 作為專用推論加速器的通則，本期未實測）。理想上，Phase 1.1 應該同時量 GPU 和 NPU，給排程器（M6）兩個 offload 選項去比較。

A5.2 為什麼是 PLACEHOLDER（佔位）？

被一個外部障礙卡住：GitHub issue #13。

Phase 0.3 量測期間，aetina 板（跑 RKNPU2 的那塊）離線了——重開機後仍連不上（連 ping 都沒回應），需要實體到場確認。所以 NPU 的 micro-benchmark（`measurements/aetina/rknpu2_matmul.json`）沒有收集到。

這是一個典型的「collect-what-you-can（能量到多少就算多少）」缺口：硬體出狀況時，我們不卡住整個 Phase 1.1，而是把量得到的（CIM/GPU/CPU/記憶體）先量完、先建模，把 NPU 標成相依項，等障礙排除再補。

A5.3 已經備妥的部分（所以補起來很快）

雖然沒上板量測，但前置作業都做完了，等 aetina 回線就能直接跑：

- 23/23 個 `.rknn` 模型已轉好（16 個投影 + 7 個 attention bmm），放在 metiscard 上。
- 上板 runner (rknnlite) 已就緒。
- 只差「把 `.rknn` 同步到 aetina → 跑 runner → 產出 `rknpu2_matmul.json`」這幾步。

換句話說，這不是「還沒開始」，而是「最後一哩卡在硬體上線」。

A5.4 對 PHASE 1.1 結論的影響（其實不大）

缺 NPU 不會動搖 Phase 1.1 的核心結論，原因：

1. Phase 1.1 是 **decode-calibrated**，主幹是 CIM（算）+ 記憶體（搬）——這兩個都齊了。
2. 「attention 該 offload」這個結論，目前站在 GPU 的數據上就足夠（A3 的 CIM 31–46ms vs GPU 幾十–幾百 μ s，差 2 個數量級，對 kernel 品質不敏感）。NPU 只是第二個佐證，不是必要條件。

NPU 補上之後會多兩樣東西：

- 第二個 **offload** 對照點（NPU 原生 attention vs GPU vs CIM penalty）。
- 「NPU 大投影不需 tiling」的對照（NPU 不像 CIM 有「輸出超過 4核 $\times$ 512=2048 就要分塊」的限制）。

這些會讓 M6 排程器（A8）在「attention 丟 GPU 還是 NPU」上有更完整的依據，但不影響 Phase 1.1 已驗證的部分。

A5.5 給 PHASE 1.2 的接口（已寫死）

我們把 NPU 的「未完成」狀態明確寫進程式與合約，讓 Phase 1.2 不會誤以為它已完成：

- `simulator/models/m4_npu.py`：是一個 stub，被呼叫就 `raise NotImplementedError`，docstring 指向 #13，並說明「等 #13 解掉，就照 A3 的 M4-GPU 方式（`FLOPs/G_eff` + 原生 attention bmm）從 `rknpu2_matmul.json` 擬合」。
- `validation/contracts/m4_npu.yaml`：`status: BLOCKED`、`blocked_on: issue #13`，並先列好待調參數（`npu_gemm_gflops`、`npu_attn_bmm_us_per_kv`）。

這就是模組化紀律：一個沒做完的元件，用「會報錯的 stub + 標清楚的合約」佔位，而不是留一個看起來能用、其實是空的東西。

一句話總結 A5：NPU 是 attention 的第二個 offload 候選，本期因 aetina 離線（issue #13）未量測；前置（23 個 `.rknn` + runner）都備妥，等板子回線即可補；缺它不動搖 Phase 1.1 的 decode 校準與「attention 該 offload」結論，只少了第二個佐證。它以「報錯 stub + BLOCKED 合約」誠實佔位。下一章 A6 回到「源頭」——看是誰決定每個 token 到底要跑哪些 op。

A6 — M5：工作負載 / trace 產生器（一切的源頭）

這一章你會學到：前面 A1–A5 都在算「某個 op 要多久」，但「一個 token 到底會跑哪些 op」是誰決定的？答案就是 M5。它跟前面元件不一樣——它沒有要預測的延遲、也沒有要擬合的參數，所以它的「驗證」是另一種：結構一致性（確認它列出的 op 不多不少，剛好等於模型真正會跑的）。

A6.1 架構考量：M5 是誰？為什麼它是「源頭」？

回顧 §0.7 的架構，M5 是最前面的「① 工作負載產生器」。它的工作：把一個 HuggingFace 模型 + 一段輸入長度，轉成「每個 token 會執行的 op 清單（含張量形狀）」。

這份清單是整條 pipeline 的輸入：

- M6 排程器拿它決定每個 op 丟給哪個單元；
- M1/M4 拿每個 op 的形狀去算延遲；
- 最後加總成端到端的 tok/s（Part B）。

所以如果 M5 列錯了 op，後面全錯。它的正確性是地基。

A6.2 原理：怎麼「不跑模型」就知道會跑哪些 OP？

關鍵洞見（§0.1 的「Phase 0.1」做的）：一個 LLM 每個 token 會跑哪些 op、形狀多大，是由模型架構 + 輸入長度「決定（deterministic）」的，跟你用什麼硬體、甚至跟權重的數值都無關。

所以我們用一個 PyTorch runtime tracer（搭配 meta / FakeTensor——一種「只有形狀、沒有真實數值」的假張量）去追蹤一次 prefill + 幾個 decode step：

- 形狀會一路傳播（matmul 的輸出形狀由輸入形狀算出），
- 但不需要載入真實權重、不需要 GPU、不需要上板——因為我們只關心「有哪些 op、形狀多少」，不關心算出什麼數值。

追蹤的結果就是 Phase 0.1 的產物：每個模型的 op inventory（distinct op 種類 + 形狀分佈 + prefill/decode 標記）。M5 在 Phase 1.1 直接重用這份東西。

A6.3 「參數設計」：M5 沒有參數

這一節很短，因為M5 是 **deterministic** 的，沒有任何要擬合的參數。給定（模型，長度），它的輸出是唯一確定的。

這帶來一個問題：沒有延遲可預測、沒有參數可擬合，那要怎麼「驗證」它？ 答案見下節——改驗「結構一致性」。

A6.4 MEASUREMENT VS PREDICTION：用「ORACLE」交叉檢查

M5 的驗證不是「量測 vs 預測延遲」，而是「兩種獨立方法數出來的 **op** 要一致」。我們有兩個獨立來源：

- 1. **runtime tracer**（實際追蹤模型跑一遍，列出 **op**）。
- 2. 架構解析式 **oracle**（ `expected_ops_check` ）：純粹照 Transformer 架構「每層該有哪些 **op** × L 層」算出來的應有清單。

如果這兩個獨立方法吻合，代表 **trace** 是對的（既沒漏、也沒多）。我們檢查兩件事：

- **語意覆蓋（semantic covered）**：oracle 列的每一種語意 **op**（8 種：matmul、attention、softmax、RMSNorm、RoPE、SwiGLU、residual、embedding）都有被 **trace** 到 → 四個模型全部 **covered = true**。（注意：下面圖 A6-1 顯示的是 9 類，那是 **op_profile** 的分法——就是這 8 種語意，再加上 `kv_cache` 這個「記憶體記帳」類；§0.3 也提過 `ffn` 這一類在 **profile** 裡指 SwiGLU。）
- **0 orphan（零孤兒）**：反過來，**op_profile** 裡出現的每一個 **op**，都必須是 **tracer** 真的產生過的（不能憑空多出一個沒被追蹤到的 **op**）。我們掃過四個模型的每一筆 **profile**：

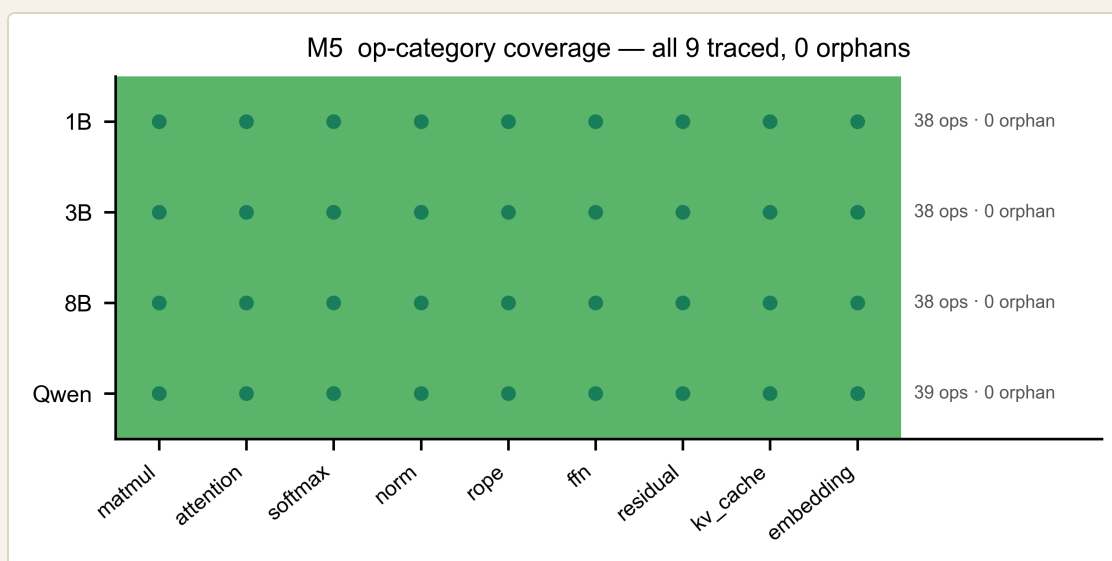
模型	DISTINCT OP 種類	PROFILE 列數	ORPHAN（孤兒）	語意覆蓋
1B	38	3267	0	✓
3B	38	3267	0	✓
8B	38	3267	0	✓
Qwen	39	3431	0	✓

0 個孤兒、全部語意覆蓋 → M5 通過。（「**profile 列數**」是每個模型 4 個任務（ShareGPT / GSM8K / HumanEval / LongBench）的 **profile** 合計，所以才有三千多列。）

一個重要的紀律：count（每個 op 跑幾次）一律取自 inventory，絕不自己手動 $\times$ layers 推算。為什麼？因為「手動乘層數」很容易出錯（GQA 的 $q \equiv o$ 、gate $\equiv$ up 共形、tied embedding 等特例），所以我們讓 tracer/oracle 自己處理，人不插手算。

A6.5 圖片解釋

圖 A6-1（M5 coverage）— op-category 覆蓋格



- 每一列 = 一個模型（1B/3B/8B/Qwen）。每一行 = 一種 op category（9 大類）。
- 每一格：綠色 ✓ = 該模型有 trace 到這一類 op。
- 怎麼看：整張格子全綠（4×9 全部 ✓）（圖中最後一列標 7b 即 Qwen-2.5-7B），標題寫「0 orphans」——代表每個模型都涵蓋全部 9 類 op，而且沒有任何一個 profile op 是「沒被追蹤到的孤兒」。這就是 M5 的「結構一致性」驗證：不多、不少、剛剛好。

A6.6 限制與 GAP (誠實清單)

項目	狀態	說明
trace 正確性	✅ 已驗證	o orphan + 語意覆蓋 + count 取自 inventory (不手動 ×L)
「meas vs pred」形式	ℹ️ 不同	M5 無延遲可比；驗的是結構一致性（兩種數法吻合）
來源	♻️ 重用 Phase 0.1	M5 前半段就是 Phase 0.1 的 runtime tracer 產物
ONNX 匯出	⚠️ 次要路徑	給 ONNXim NPU 用的 ONNX 是次要產物、有 fallback;主路徑用 runtime tracer (不靠 ONNX)

一句話總結 A6：M5 是整條 pipeline 的源頭，它用「不載權重的形狀追蹤」決定每個 token 會跑哪些 op;因為它 deterministic、沒有參數，所以驗的不是延遲而是結構一致性——用架構 oracle 交叉檢查，四個模型 o 孤兒、9 類全覆蓋，且 count 絕不手動 ×層數。到這裡 Part A 把「可量測校準」的元件 (M1/M2/M4/M5/M7) 都走完了；A7 補上能耗，A8 講「為什麼 M3/M6 這兩個整合層只定合約」。

A7 — M7：能耗估算（沒有電表，怎麼談省電？）

這一章你會學到：為什麼這個專案量不到真實功耗、那我們怎麼還能談能耗、以及一個強力支持 CIM-centric 主張的結果——decode 的能量幾乎全花在「搬權重」，CIM 的「算」便宜到可以忽略。而且這個結論禁得起參數 $\pm 20\%$ 的搖晃。

A7.1 架構考量：M7 是誰？為什麼它最特別？

M7 是「⑤ 能耗估算」（§0.7）。前面 M1/M2/M4 算「多快」，M7 算「多耗電」（每個 token 花多少焦耳）。

但 M7 有一個獨特的困難：兩塊板子都沒有可用的功耗量測。

- Aetina Metis Alpha：只有廠商的 TDP 估計，沒有逐 op 的功耗。
- 量產 Metis Card：是 PCIe Rev1 測試板，沒有功耗 telemetry。

所以我們根本量不到真實能耗。這跟前面所有元件不一樣——M1 有量測可比，M7 沒有。這就逼出一個誠實的問題：沒有電表，怎麼談省電？

A7.2 原理：用「規格」估算，不用「量測」

既然量不到，我們改用規格（spec）估算（這是 ADR-0005 定的策略）：每個單元的能量 = 它的「活動量」× 它的「能效規格」。

CIM 計算能量 = 運算數 ÷ (15 TOPS/W)	← 廠商 Metis INT8 能效
DRAM 搬移能量 = bits × 4 pJ/bit	← JEDEC LPDDR5 每 bit 存取
PCIe 搬移能量 = bits × 5 pJ/bit	← PCIe 規格
CPU 計算能量 = 0.75 W/核 × 4 核 × 活動時間	← ARM A76 datasheet

兩個關鍵單位（給初學者）：

- TOPS/W（tera-ops per watt）：每瓦能做幾兆次運算 = 每焦耳能做幾兆次運算。15 TOPS/W 代表「1 焦耳能做 15 兆次運算」→ 運算越多越耗能，但 CIM 很省。
- pJ/bit（picojoule per bit）：搬 1 個 bit 要花幾皮焦耳。LPDDR5 約 4 pJ/bit、PCIe 約 5 pJ/bit。

這些常數是規格/假設，已標註來源：15 TOPS/W 是 Metis 廠商值；4、5 pJ/bit 是 JEDEC/PCIe 規格的代表值（假設值，明標）；0.75 W/核 是 A76 datasheet。我們不假裝這些是量出來的。

A7.3 參數設計

公式很直接（見上），參數就是那 5 個規格常數。把它們套到 8B 模型一個 **decode token** 的活動上：

- CIM 計算：一個 token 要做的乘加 $\approx$ 權重參數量（GEMV 每個權重做一次乘加） $\approx 7.5 \times 10^9 \rightarrow$ 運算數 $= 2 \times \approx 1.5 \times 10^{10} \rightarrow \div 15e12 = 1.0 \text{ mJ}$ 。
- DRAM 搬移：要串 7.5 GB 權重 $= 6 \times 10^{10} \text{ bits} \times 4 \text{ pJ} = 240 \text{ mJ}$ 。
- CPU 支援：所有輔助 op（A4 的 rmsnorm/rope/swiglu/softmax/sampling $\times$ 層數）的活動時間 $\times$ 功率 $\approx 15 \text{ mJ}$ 。

A7.4 「MEASUREMENT VS PREDICTION」：沒有量測，改驗「穩健性」

這是 M7 最誠實的地方：沒有真實能耗可比，所以沒有傳統的量測 vs 預測。那要怎麼相信這個估算？我們改用三道穩健性檢查：

1. **Sanity**（合理性）：能量為正、隨活動量單調增加。✅
2. 獨立量級檢查：把估出的每 token 總能量（256 mJ） $\div$ 量到的 decode 時間（8B $\approx 1/2.70 = 0.37 \text{ s}$ ） $\rightarrow$ 推得平均功耗約 **0.69 W**。這對一顆行動 SoC 是合理的（落在 0.1–20 W），所以量級沒錯得離譜。（注意：這用到了「量測到的 decode 速度」當獨立參照，不是自己對自己。）
3. $\pm 20\%$ 敏感度（最重要）：把那 5 個規格常數各自 $\pm 20\%$ （16 種組合都試），檢查「結論會不會翻盤」。我們關心的定性結論是「**decode 的能量由記憶體主導**」——結果 16 種組合全部成立、0 次翻盤。也就是說，就算我們的 pJ/bit、TOPS/W 都抓錯 20%，「memory 主導」這個結論依然站得住。

核心結果（也是 CIM-centric 的有力證據）：

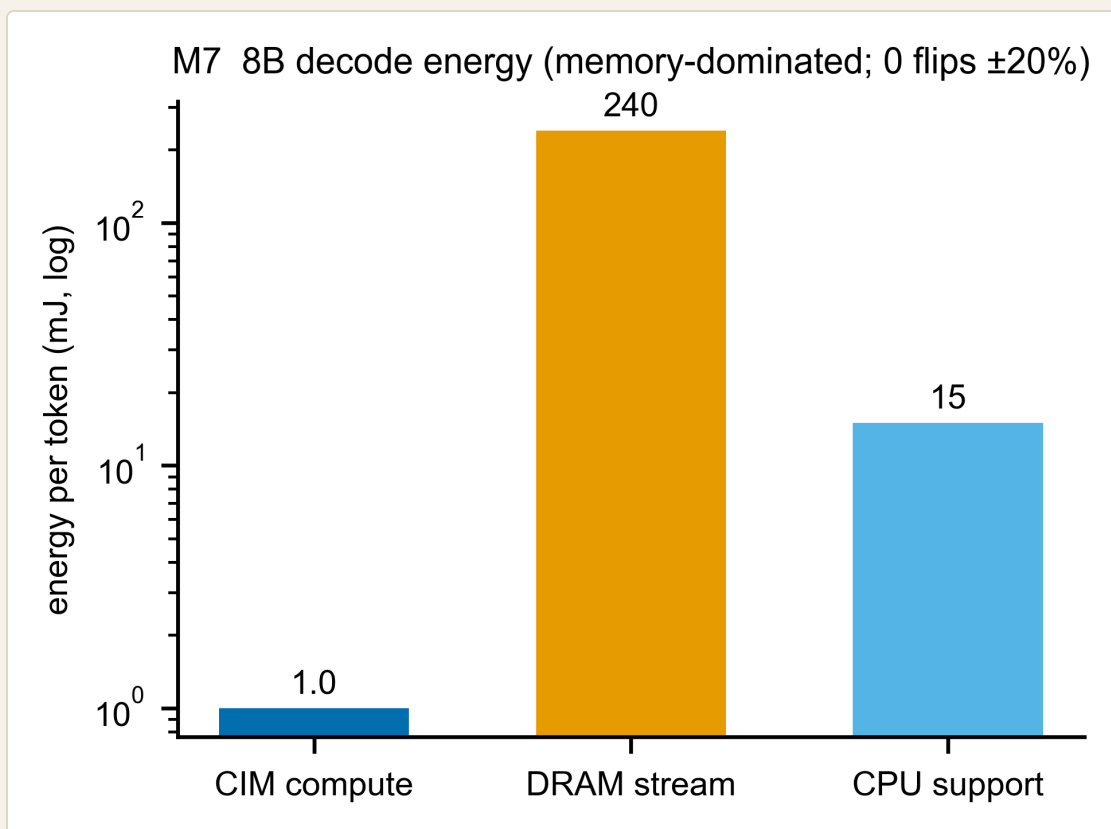
8B DECODE 每 TOKEN	能量	佔比
CIM 計算（算 matmul）	1.0 mJ	0.4%
DRAM 搬移（串權重）	240 mJ	94%
CPU 支援	15 mJ	6%

CIM 的「算」只花 1 mJ，搬權重花 240 mJ——差 240 倍！這正是「memory wall（記憶體牆）」的能量版本，也精準呼應 §0.4 的 CIM-centric 主張：CIM 計算又快又省，瓶頸（時間和能量）都在搬資料。系統該繞著「減少搬移」設計，而不是「加速計算」。

為什麼分解裡看不到 PCIe？A7.2 的公式列了 PCIe（5 pJ/bit），但 8B decode 的三根長條沒有它——因為這裡的「搬權重」是從 on-package DRAM 串流，不走 PCIe（正是 A2 講的 floor 適用邊界：decode 串流不過 PCIe）。PCIe 能量只在「離散 host<=>device 搬移」時才出現，那是 Phase 2 整合時的事。

A7.5 圖片解釋

圖 A7-1（M7 energy）— 8B decode 每 token 能量分解



- X 軸：三個來源（CIM 計算 / DRAM 搬移 / CPU 支援）。Y 軸：每 token 能量（mJ，對數軸）。
- 怎麼看：用對數軸是因為差距太大——DRAM（240）的長條比 CIM（1.0）高出兩個數量級。一眼就看出「能量幾乎全在搬資料」。標題註明「memory-dominated；0 flips $\pm 20\%$ 」提醒這個結論對參數抖動穩健。

A7.6 限制與 GAP（誠實清單）

項目	狀態	說明
能耗本身	⚠️ 估算非量測	兩板皆無功耗 telemetry（ADR-0005）；用規格估算
結論可信度	✅ 穩健	「memory 主導」對所有規格常數 $\pm 20\%$ 不翻盤（16/16）
規格常數	✎ 假設	15 TOPS/W、4/5 pJ/bit、0.75 W/核——規格/假設值，明標來源
CPU 支援時間	⚠️ 粗估	每 token 的 A76 活動時間是粗略估計

方法學定位：最接近的競品 HPIM 完全不報能耗；CENT/PAPI 用 datasheet 估算。所以一個「透明標註 + 帶 $\pm 20\%$ 敏感度」的規格模型，是符合（甚至優於）領域慣例的。

一句話總結 A7：因為兩板都沒電表，M7 用規格估算而非量測，所以驗的不是準確度而是穩健性（ $\pm 20\%$ 不翻盤）；最重要的發現是 decode 能量 94% 花在搬權重、CIM 計算只佔 0.4%——這是 CIM-centric「瓶頸在記憶體」主張的能量證據。Part A 到此把所有「可量測校準」的元件走完；最後 A8 解釋為什麼 M3、M6 這兩個整合層在 Phase 1.1 只定合約、不實作。

A8 — M3 / M6：兩個「只能定合約、還不能驗證」的整合層

這一章你會學到：為什麼有兩個模組（M3 事件引擎、M6 排程器）在 Phase 1.1 故意不實作、只寫一份「合約」；以及一個直接打到專案招牌的誠實揭露——混合精度的轉換成本，當初說好要量，但根本沒量到。

A8.1 架構考量：M3、M6 跟前面哪裡不同？

前面 A1–A7 的元件（M1/M2/M4/M5/M7）有個共同點：它們都能「單獨」量測或驗證——M1 量一個 matmul、M4 量一個 attention，各自獨立。

但 M3 和 M6 不一樣，它們是整合層（把別人串起來的膠水）：

- **M3 — 事件引擎（event engine）**：把一個 token 的 op 流，實際串過各個單元 + 記憶體，模擬「誰先誰後、誰跟誰搶頻寬」，算出端到端時間。
- **M6 — 排程器 / 對應器（scheduler / mapper）**：決定每個 op 丟給哪個單元、用什麼精度、放哪塊記憶體。這是整篇論文的貢獻層（§0.4 的 CIM-centric 分工就是它在做）。

關鍵差異：你沒辦法「單獨」驗證一個排程器或一個事件引擎——你得先把它實作出來、真的把整個 LLM 跑一遍，才知道它對不對。而「實作 + 跑端到端」正是 **Phase 2** 的工作。

A8.2 為什麼 PHASE 1.1 只定合約、不實作？

這是 Phase 1.1 vs Phase 2 的分工（§0.6）：

- **Phase 1.1 = 單一元件校準**：每個能獨立量的元件，校到準（A1–A7）。
- **Phase 2 = 整合**：把校好的元件組成模擬器，M3/M6 才在這時被實作 + 端到端驗證。

所以 Phase 1.1 對 M3/M6 做的，是寫一份驗證合約（contract）：白紙黑字定好「之後 Phase 2 實作時，要達到什麼標準、有哪些參數要調」。這份合約就是 Phase 1.1 給 Phase 2 的交棒規格。

把它想成蓋房子：A1–A7 是「先確認每一種建材的強度」，M3/M6 是「整棟的結構設計」——結構要等建材到齊、真的蓋起來才測得出穩不穩，但你現在就能先把驗收標準寫進合約。

A8.3 M3 合約：事件引擎要驗什麼

M3 是一個輕量級的離散事件引擎（ADR-0001 定的 fidelity：不做 cycle-level，但要模擬頻寬競爭）。合約（`validation/contracts/m3.yaml`）寫了：

- 驗收標準（Phase 2 系統級門檻，ADR-0006）：端到端 latency/tok-s 誤差 $\leq 15\%$ ；頻寬競爭拐點 $\leq 15\%$ 。
- 待調參數：
- `bandwidth_contention_knee` ——這是 ADR-0001 的硬性義務：要能重現「多個單元一起搶記憶體時，總頻寬在共享峰值之下飽和」的拐點（這是 memory-wall 系統行為的關鍵）。（數字怎麼來：HeteroInfer 在它的手機 SoC 上量到 GPU+NPU ≈ 60 GB/s、低於該平台 68 GB/s 峰值，這是個說明用的參照；本專案自己的飽和拐點是由 Aetina 的並行 micro-benchmark 校準的，不是直接套 60 GB/s。）
- `interconnect_efficiency`、`concurrency_overlap_factor`（CIM // GPU // NPU 的重疊程度）。

為什麼頻寬競爭這麼重要？因為 CIM-centric 的賣點之一是「異質單元並行 + 共享記憶體」。如果各單元一起跑、把記憶體頻寬吃爆，並行的好處就打折——這個「拐點」抓不準，整個系統預測就不可信。所以 ADR-0001 把它列為必驗。

A8.4 M6 合約：排程器要驗什麼（含招牌貢獻）

M6 是貢獻層。合約（`validation/contracts/m6.yaml`）寫了：

- 驗收標準：端到端 $\leq 15\%$ ；SOTA 重現（能重現已知的分工策略結果）。
- 待調參數：`op_to_unit_mapping_policy`（op→單元）、`precision_boundary_placement`（精度邊界放哪）、`memory_placement`，以及——最關鍵的 `precision_boundary_conversion_op_cost`。

最後這個參數，牽出 Phase 1.1 最重要的一個誠實揭露，見下節。

A8.5 招牌貢獻的破洞：混合精度的「轉換成本」沒量到

這是必須講清楚的一件事。

背景：這個專案的主要研究面是混合精度——CIM 用 INT8、GPU 用 FP16,各單元原生精度不同。當資料從一個精度的單元流到另一個（例如 CIM-INT8 的 FFN 輸出 → GPU-FP16 的 attention),中間要插一個轉換 op（quant/dequant,量化/反量化），這要花時間和能量。

問題：ADR-0004 當初白紙黑字說「這個轉換 op 的成本會在 Phase 0.2 校準」。但是：

- 轉換 op 不在 9 大類 op 裡——因為它是排程器在精度邊界「插入」的，不會出現在 HuggingFace 模型的原始 trace 裡（\$0.3/A6 列的那 9 類是模型本身的 op）。
- `grep quant measurements/` = 0 筆;cpu_ops 裡也沒有量化 op。
- 講好的 Phase 0.2 校準，從來沒發生。

後果：Phase 2 的 M6 排程器會在精度邊界插入轉換 op,但沒有任何地方給它成本——專案的招牌貢獻（混合精度),目前跑在一個沒被計價的 op 上。

我們怎麼誠實處理? 把它明確寫進 M6 合約：`precision_boundary_conversion_op_cost` 列為待調參數 + 量測 gap（繫 ADR-0004），並註明「Phase 2 必須先量 dequant/requant（便宜的逐元素 op,可在 CPU/NPU 量）,混合精度的主張才站得住」。這樣這個洞就不會默默漏到 Phase 2、直到最後才爆。

A8.6 限制與 GAP（誠實清單）

項目	狀態	說明
M3 行為驗證	⏸ Phase 2	需先實作事件引擎才能測；Phase 1.1 只定合約
M6 行為驗證	⏸ Phase 2	需先實作排程器才能測；Phase 1.1 只定合約
頻寬競爭拐點	🔴 待驗	ADR-0001 硬性義務：重現共享頻寬飽和拐點（HeteroInfer 60 GB/s 為參照，本專案由 Aetina 校準);列入 M3 合約
轉換 op 成本	❌ 量測 gap	ADR-0004 說好的 Phase 0.2 校準沒做;招牌混合精度貢獻無成本基礎；列入 M6 合約待 Phase 2 補量

一句話總結 A8：M3(事件引擎)和 M6(排程器)是整合層，沒實作就驗不了，所以 Phase 1.1 只給它們寫驗收合約交棒給 Phase 2;過程中我們誠實揭露了一個打到招牌的破洞——混合精度的轉換 op 成本，ADR-0004

說好要量卻沒量，已明確列入 M6 合約當 Phase 2 的必補項。Part A 到此結束;接下來 Part B 把這 7 個校好的零件組起來，真正去預測一個 LLM 的 decode 速度。

Part B — 把零件組起來：預測一個真實 LLM 的 decode 速度

這一章你會學到：Part A 把每個零件（M1–M7）各自校準好了；Part B 是驗收的時刻——把它們組起來，去預測一個真實 LLM 跑得多快（tok/s），再跟量產 Metis Card 的實測比。這是整個 Phase 1.1 唯一的「端到端」考試，也是「零件對 → 整機對」的關鍵一步。

B.1 為什麼需要這一步？（零件對不代表整機對）

Part A 證明了每個零件準（M1 的 CIM 吞吐 2.7%、M4-GPU 的 attention 0.6%...）。但零件準 ≠ 組起來準——組合時可能漏項、可能重複計算、可能拓模假設錯。所以我們要做一个端到端的重新合成（recompose）：用校好的公式 + Phase 0.2 的 op 次數，去預測一個真實 LLM 的 decode 速度（tok/s），再跟真晶片比。

這也呼應 §0.8 的兩種門檻：這裡用的是較寬鬆的端到端門檻 $\leq 25\%$ （因為疊了很多元件 + 拓模假設）。

B.2 DECODE 速度怎麼預測？（一條極簡的公式）

回顧 §0.3 + A2 + A7 的核心事實：decode 是 memory-bound，瓶頸是「把權重從記憶體串出來」。所以：

$$\begin{aligned} \text{decode 速度 (tok/s)} &= \text{有效記憶體頻寬} \div \text{每個 token 要串的權重位元組} \\ \text{tok_s} &= \text{BW_eff} \div \text{weight_bytes} \end{aligned}$$

就這麼簡單。一個 8B 模型每 token 要串 7.5 GB（A7 算過），所以它一定比 1B（1.24 GB）慢——因為要搬的權重多 6 倍。

weight_bytes 怎麼來的？由 Phase 0.2 的 op_profile 逐 op 累加 decode 階段的 matmul 位元組得到（這是 Phase 1.1 對舊方法的精化）。算出來：1B = 1.24 GB、3B = 3.21 GB、8B = 7.51 GB——和理論值吻合到 0.1%。

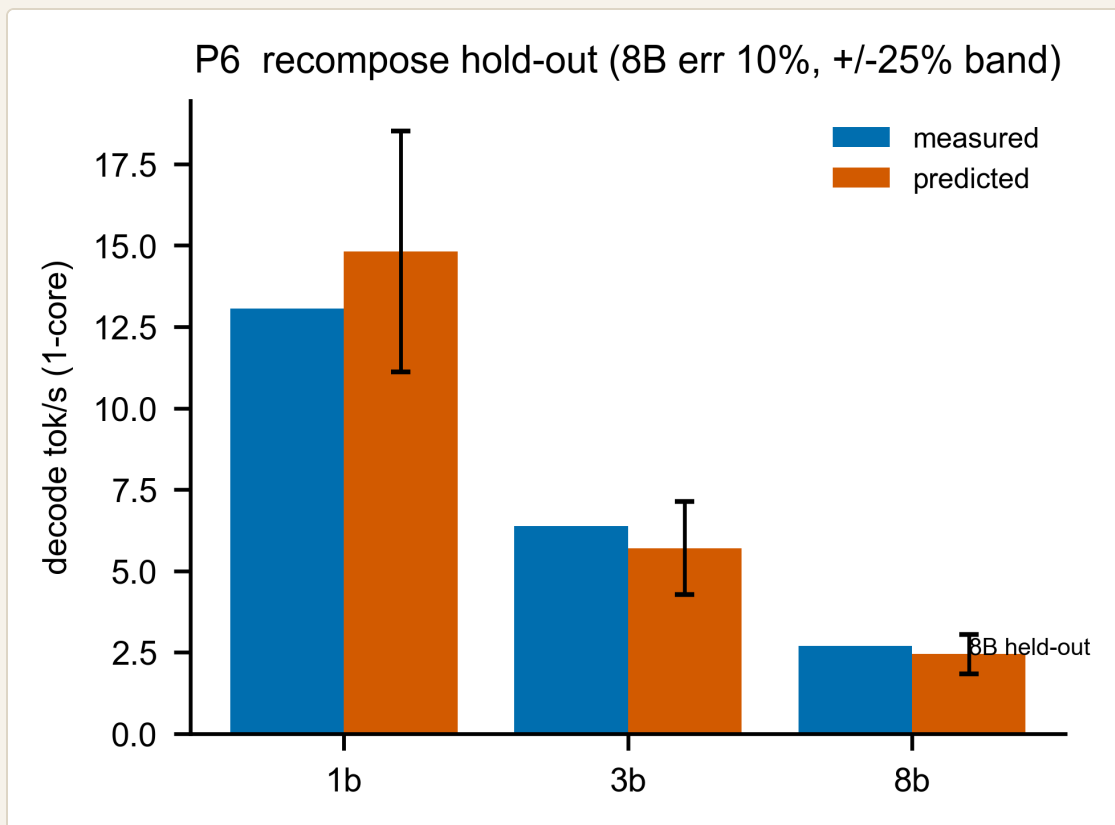
B.3 MEASUREMENT VS PREDICTION : 8B 的「保留考試」

這是 Phase 1.1 最重要的一個驗證，而且我們刻意讓它非循環（\$0.5 的 held-out）：

1. 只用 1B 和 3B 的實測 tok/s, 反推出有效頻寬 BW_{eff} 。算出 $BW_{eff} = 18.33 \text{ GB/s}$ 。
2. 完全不看 8B，直接用這個 BW 去預測 8B: $pred_8B = 18.33 \text{ GB/s} \div 7.51 \text{ GB} = 2.44 \text{ tok/s}$ 。
3. 再跟 8B 的實測比：實測 $2.70 \text{ tok/s} \rightarrow$ 誤差 9.5%，遠在 25% 門檻內。✅

為什麼這很有力？因為 8B 是「考題」——它的答案完全沒參與擬合。模型在「沒看過的模型大小」上預測準，代表它真的抓到了 **memory-wall** 的規律，不是硬湊。

圖 B-1 (P6) — 端到端 recompose 量測 vs 預測



- X 軸：模型（1B / 3B / 8B）。Y 軸：decode 速度（tok/s，1 核）。
- 藍 = 實測，橘 = 預測，橘色有 $\pm 25\%$ 誤差帶。
- 怎麼看：1B/3B 是「拿來擬合 BW」的點（所以橘藍接近）；8B 是 held-out 考題，橘色預測（2.44）落在實測（2.70）的 $\pm 25\%$ 帶內 $\rightarrow$ 通過。

一個有趣的細節（size 趨勢）：各模型反推的有效頻寬是 $16.2 / 20.5 / 20.3 \text{ GB/s}$ (1B/3B/8B)。可以看到小模型的有效頻寬較低（串權重較沒效率）——這是真實的 size 依賴，也正是為什麼我們用 1B+3B $\rightarrow$ 8B 的 hold-out 而不是自己對自己。

B.4 拓樸的誠實：為什麼 ALPHA 的 911MS FLOOR 不進這個預測

A2 講過 Alpha 板每次 decode 都付 911 μ s 的 PCIe floor。但這個預測裡故意不放它，為什麼？

因為我們預測的對象是量產 Metis Card（有 on-card DRAM），decode 串流權重時權重就在卡上、不走 PCIe → 不付那筆 floor。Alpha 的 911 μ s 是該板沒有 on-card DRAM 的拓樸限制，findings 明禁把它外推到量產卡。這就是 §0.4 講的「橋接假設」：CIM 計算 timing 不變，只有資料搬移 timing 隨拓樸改變。

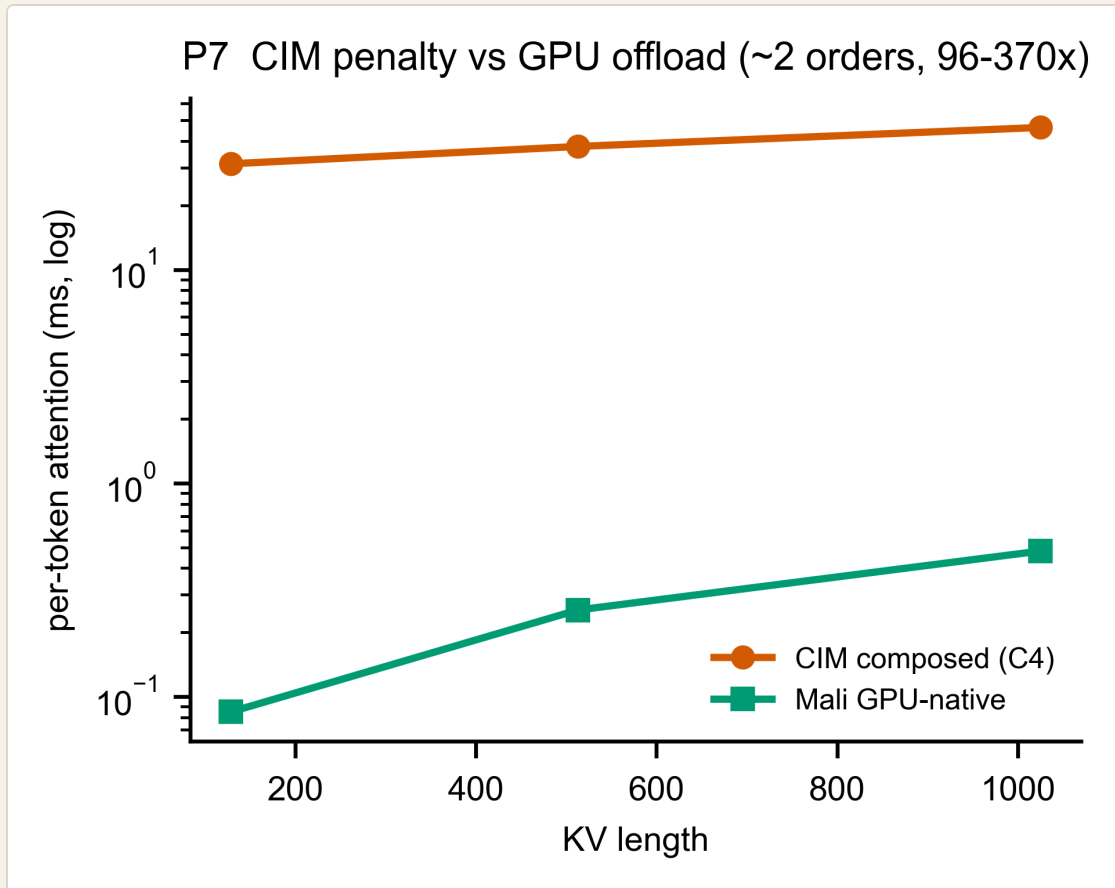
B.5 為什麼 ATTENTION 一定要 OFFLOAD? (C4 的量化證據)

A3 說「CIM 不擅長 attention,要丟給 GPU」。Part B 給出量化證據。我們組合出「如果硬要 CIM 做 attention」的成本 (C4):每個 decode step,CIM 得把長大的 KV-cache 重新載入 crossbar,成本是：

$$\text{CIM attention} \approx \text{每層(KV 重載 bytes} \div \text{頻寬} + \text{固定 floor)} \times \text{層數} \approx 31\text{--}46 \text{ ms/token}$$

對比 GPU 原生 attention(A3):幾十到幾百 μ s。差約 2 個數量級 (96–370 倍)!

圖 B-2 (P7) — CIM attention penalty vs GPU offload



- X 軸：KV 長度。Y 軸：每 token 的 attention 延遲（ms，對數軸）。
- 橘 = CIM composed（含 KV 重載）、綠 = Mali GPU 原生。
- 怎麼看：對數軸下兩條線差了約 2 格（兩個數量級，96–370×）。把 attention 留在 CIM 會讓每個 token 多付 31–46 ms 的 KV 重載 penalty，相對 decode 主幹（幾百 ms）是一筆可觀（約 8–12%）、但短上下文下還不致翻盤 gate 的額外負擔（長上下文下會更糟，因為 KV 重載隨上下文成長）。

注意這張圖的「比較單位」：橘線（CIM）是整個 token、所有層的 composed penalty（主要是 KV 重載），綠線（GPU）是單一 head 的原生計算——所以這是一個「為什麼要 offload」的示意對照，不是嚴格的同單位 head-to-head。它要傳達的結構性事實才是重點：CIM 每個 decode step 都得把長大的 KV 重載進 crossbar（一筆隨上下文成長的頻寬 penalty），而 GPU/NPU 原生做 attention 不需要這個重載。這個結構差異不因 kernel 好壞而消失，所以「offload」結論穩固。另外這個 C4 值是 Alpha 拓樸的上界估計，且不放進 B.3 的 decode 預測(那裡 attention 走 GPU-offload)。

B.6 那些「沒進主預測」的項（誠實揭露）

B.3 的 decode 預測只用了 weight-streaming 主幹(BW_eff/weight_bytes)。但一個 token 其實還有別的成本：CPU 支援 op、GPU-offload 的 attention、kv_cache append。為什麼沒加進去？

因為 BW_eff 是從實測 tok/s 反推的，它已經「吸收」了這些成本（實測時間本來就包含它們）。如果再把它們額外加上去，就會重複計算（double-count）。所以我們把這些項單獨列出來當透明參考，但不加進主預測：

8B DECODE 每 TOKEN（單獨列，未加總）	時間
decode 串流主幹	~409 ms
CPU 支援 op	~62 ms
GPU-offload attention（未優化 kernel × heads × layers）	~260 ms
kv_cache append	~1.4 ms

⚠ 別把這裡的 260 ms 跟 B.5 的 CIM 31-46 ms 直接比、誤以為「CIM 比較快」——兩個是不同的量，而且這個 260 ms 是個寬鬆的上界：它是 A3 那顆未優化 GPU kernel 的單-head 成本再 ×(heads × layers) 堆出來的（heads×layers 的聚合方式本身還是 Phase-2 的 watch-item）。一顆調教過的 Mali kernel 會遠低於此。（補充：recompose 的 JSON 欄位把它叫 lower_bound，指的是吞吐的下界——吞吐下界等價於延遲的上界，跟這裡講「260 ms 是寬鬆上界」是同一回事。）B.5 的「該 offload」結論靠的是結構性論證（CIM 有「KV 每步重載」的頻寬 penalty,GPU/NPU 原生做 attention 沒有這個 penalty）,不是這個被 kernel 品質灌水的絕對值。

另外這也是記在案的 Phase-2 watch-item：這些項沒有加進 B.3 的主預測，因為 BW_eff 是從實測反推、已經吸收了它們，再加就重複計算；短上下文下它們小、不影響 8B 的 gate,但長上下文（LongBench）下 kv_cache 會變大，Phase 2 要決定它是「加性項」還是「已含在頻寬裡」。

B.7 PREFILL: 誠實標為「未驗證」

整個 Part B 講的都是 decode。prefill（預填）我們明確標為未驗證，原因（A1/A4 都提過）：

- CIM 的 prefill 投影 (M≥512) 在裝置上配置失敗，沒量到。
- prefill 的 attention、softmax 是 S×S 形狀，本期沒涵蓋。

我們有量產卡的 prefill 時間錨點（`ttft_s_median`, 8B = 3.79 s）。用它反推，**prefill** 的 GEMM 吞吐約 4.1 TOPS——但如果錯用 decode 的吞吐（204 GOP/s）去估，會得到荒謬的 75 秒。這個 20 倍的差距正好證明：**prefill** 的吞吐和 **decode** 完全不同，而我們沒量到它。所以 prefill 預測是 **best-effort**、不評分、列為 Phase 2 必補。

（順帶：vendor 還有一個 `prefill_ms_median` 欄位，但它跨模型都是 ~0.007 s 不變，是個壞掉的 degenerate 欄位，我們棄用、改用會隨模型變的 `ttft_s_median`。）

B.8 PART B 總結

項目	結果
decode 端到端（唯一硬 gate）	✅ 8B held-out 2.44 vs 2.70 = 9.5%(≤25%)
weight_bytes 來源	op_profile 逐 op 累加，對理論值 0.1%
size 趨勢	有效頻寬 16.2/20.5/20.3 GB/s（小模型較低）
attention offload 證據	CIM 31–46ms vs GPU 幾百μs, 差 2 個數量級
非串流項	單獨列、未加總（避免 double-count, Phase-2 watch-item）
prefill	❌ 未驗證（裝置配置失敗；best-effort、不評分）

一句話總結 **Part B**：把 Part A 的零件組起來，用「頻寬 ÷ 權重位元組」這條極簡公式，在沒看過的 8B 模型上預測 decode 速度，誤差 9.5%——證明「零件對 → 整機對」；同時量化證明 attention 該 offload(差 2 個數量級)、誠實揭露 prefill 未驗證與 double-count 的 Phase-2 待辦。這就是 **Phase 1.1** 的終點：一組校準好、誠實標註、可外推的 **component** 模型，交給 **Phase 2** 組成完整模擬器。